

Contents

1 Basic	1
1.1 Shell Scripts	1
1.2 Debug	1
1.3 Int128	1
1.4 Div	1
1.5 Pbds and Cmp	1
2 Graph	2
2.1 Binary Lifting LCA	2
2.2 Heavy Light Decomposition	2
2.3 Centroid Decomposition	2
2.4 SCC	2
2.5 2-SAT	3
2.6 EBCC	3
2.7 VBCC	3
2.8 Functional Graph	3
2.9 Euler	4
2.10 Virtual Tree	4
2.11 Dominator Tree	4
3 Data Structure	4
3.1 Fenwick	4
3.2 Lazy Segment Tree	5
3.3 Dynamic Segment Tree	6
3.4 Persistent Segment Tree	6
3.5 Kth-element	6
3.6 Link Cut Tree	6
3.7 LCT Extension	7
3.8 Treap	7
3.9 KDTree	8
3.10 Sparse Table	8
3.11 Mo	8
3.12 Rollback DSU	8
4 Flow Matching	9
4.1 Dinic	9
4.2 Min Cut	9
4.3 MCMF	9
4.4 Hungarian	10
5 String	10
5.1 Hash	10
5.2 KMP	10
5.3 Z Function	10
5.4 Manacher	10
5.5 Trie	10
5.6 SA	11
5.7 AC	11
5.8 SAM	11
5.9 Palindrome Tree	12
5.10 Duval	12
6 Math	12
6.1 Mint	12
6.2 Sieve	12
6.3 Matrix	13
6.4 Miller Rabin Pollard Rho	13
6.5 Primitive Root	13
6.6 CRT	13
6.7 XOR Basis	13
6.8 Gaussian Elimination	14
6.9 Integer Partition	14
6.10 Basic Theorems	14

6.10.1 Combination	14
6.10.2 Game Theorem	14
6.10.3 Pisano Period	14
6.10.4 Burnside's lemma	14
6.10.5 Catalan Theorem	14
6.10.6 Inversion	15
6.11 Mobius Inverse	15
6.12 LCM Convolution	15
6.13 exLucas	15
6.14 Quadratic Residue	15
6.15 BSGS	15
7 Search and Greedy	16
7.1 Binary Search	16
7.2 Ternary Search	16
8 DP	16
8.1 Projects	16
8.2 Bitmask	16
8.3 Monotonic Queue	16
8.4 Digit	16
8.5 SOS	17
8.6 CHT	17
8.7 DNC	17
8.8 LiChao Segment Tree	17
9 Geometry	18
9.1 Basic	18
9.2 Sweep And Polar Sort	18
9.3 Convex Hull	18
9.4 Distance	18
9.5 Point In Polygon	18
9.6 Point In Convex Polygon	18
9.7 Line Intersects Polygon	19
9.8 Segment Intersection	19
9.9 Segment In Polygon	19
9.10 Half Plane Intersection	19
9.11 Min Euclidean Distance	19
9.12 Max Euclidean Distance	20
9.13 Min Circle Cover	20
9.14 Min Rectangle Cover	20
9.15 Convex Polygon Distance	20
9.16 Lattice Points	20
9.17 Polygon Union Area	20
10 Polynomial	21
10.1 FFT	21
10.2 NTT	21
10.3 FWT	21
10.4 Polynomial	21
10.5 Barlekamp Massey	22
10.6 Linear Recurrence	22
11 Else	23
11.1 Segment Manipulation	23
11.2 Python	23
11.3 Bigint	23
11.4 Multiple	23
11.5 Division	24
11.6 Division-Python	24

```

cmp $CODE1.out $CODE2.out || break
done
# 多重解, ifstream in(argv[1]);
CODE="a"
set -e
g++ $CODE.cpp -o $CODE
g++ gen.cpp -o gen
g++ checker.cpp -o checker
for ((i=1;;i++))
do
./gen > input
./$CODE < input > $CODE.out &
./checker $CODE.out < input || break
# 互動
CODE="a"
set -e
g++ $CODE.cpp -o $CODE
g++ checker.cpp -o checker
PIPE_IN="in"
PIPE_OUT="out"
trap 'rm -f $PIPE_IN $PIPE_OUT' EXIT
mkfifo $PIPE_IN $PIPE_OUT
for ((i=1;;i++))
do
echo "--- Testing: Case #${i} ---"
./$CODE < $PIPE_IN > $PIPE_OUT &
(exec 3>$PIPE_IN 4<$PIPE_OUT; ./checker <&4 >&3) || break
done
# 參考 checker
ll AC(int n) { return ans; }
void WA(string log = "") { cerr << log << endl; exit(1); }
void checkAC(string log = "") {
string trash;
if (cin >> trash) WA("redundant output\n" + log);
}
int main() {
int n = uniform_int_distribution<int>(1, 10)(rng);
ll sol = AC(n);
stringstream log;
cout << n << endl;
log << n << endl;
log << "judge: " << endl;
log << "team: " << endl;
WA(log.str());
checkAC();
return 0;
}

```

1.3 Int128 [eb585]

```

using i128 = __int128_t; // 1.7E38
istream &operator>>(istream &is, i128 &a) {
i128 sgn = 1; a = 0;
string s; is >> s;
for (auto c : s) {
if (c == '-') sgn = -1;
else a = a * 10 + c - '0';
}
a *= sgn;
return is;
} // 7ea35c
ostream &operator<<(ostream &os, i128 a) {
if (a == 0) return os << 0;
string res;
if (a < 0) os << '-', a = -a;
while (a) res.push_back(a % 10 + '0'), a /= 10;
reverse(res.begin(), res.end());
os << res;
return os;
} // e5d7e9

```

1.4 Div [5a160f]

```

template<class T> T floor(T a, T b)
{ return a / b - (a % b != 0 && ((a < 0) != (b < 0))); }
template<class T> T ceil(T a, T b)
{ return a / b + (a % b != 0 && ((a < 0) == (b < 0))); }
// a / b < x -> floor(a, b) + 1 <= x
// a / b <= x -> ceil(a, b) <= x
// x < a / b -> x <= ceil(a, b) - 1
// x <= a / b -> x <= floor(a, b)

```

1.5 Pbds and Cmp [6e57d7]

```

// pbds_multiset:
// 必須 = , 其他 STL 都不能
// lower_bound, upper_bound 交換用法
// 不要用 find(x), erase(x), 用 upper_bound 拿到 it 再判斷
// order_of_key(x) 幾個元素嚴格小於 x
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template<class T, class F = less<T>>
using pbds_set = tree<T, null_type
, F, rb_tree_tag, tree_order_statistics_node_update>;
template<class T, class F = less_equal<T>>
using pbds_multiset = tree<T, null_type
, F, rb_tree_tag, tree_order_statistics_node_update>;
// priority queue 小到大是 >, set 是 <

```

1 Basic

1.1 Shell Scripts [54e75c]

```

# ~/.bashrc
CXX="g++"
std=c++17 -Wall -Wextra -g -fsanitize=address,undefined"
run() {
local b=./$(hash $1).out
[ -x $b ] || $CXX $1 -o $b && $b "${@:2}"
}
hash() {
cpp $1 -dD -
P -fpreprocessed | tr -d "[:space:]" | md5sum | cut -c -6
}

```

1.2 Debug [33ccce]

```

# 對拍
CODE1="a"
CODE2="ac"
set -e
g++ $CODE1.cpp -o $CODE1
g++ $CODE2.cpp -o $CODE2
g++ gen.cpp -o gen
for ((i=1;;i++))
do
echo "--- Testing: Case #${i} ---"
./gen > input
# python3 gen.py > input
./$CODE1 < input > $CODE1.out
./$CODE2 < input > $CODE2.out

```

```

auto cmp = [](int i, int j) { return i > j; };
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);

vector<int> a{1, 2, 5, 4, 3}; // 小心不要改到 a
auto cmp = [&a](int i, int j) { return a[i] > a[j]; };
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);

vector<int> v{1, 2, 3, 4, 5};
upper_bound(v.begin(), v.end(), 2, [](int a, int b)
{ return a < b; }); // find first b that a < b, a is 2
lower_bound(v.begin(), v.end(), 2, [](int a, int b)
{ return a < b; }); // find first a that a < b fail, b is 2

```

2 Graph

2.1 Binary Lifting LCA [5438e3]

```

const int N = 2E5;
const int Lg = __lg(N); // __lg(max(N, Qi)), [0, Lg]
int up[N][Lg + 1];
vector<int> dep, dfn;
void build(int n, vector<vector<int>> &g, int rt = 0) {
    dep.assign(n, 0), dfn.assign(n, 0);
    int cur = 0;
    [&](this auto &&self, int x, int p) -> void {
        dfn[x] = cur++;
        up[x][0] = p;
        for (int i = 1; i <= Lg; i++) {
            int nxt = up[x][i - 1];
            up[x][i] = up[nxt][i - 1];
        }
        for (auto y : g[x]) {
            if (y == p) continue;
            up[y][0] = x;
            dep[y] = dep[x] + 1;
            self(y, x);
        }
    } (rt, rt);
}
int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a, b);
    int pull = dep[a] - dep[b];
    for (int i = 0; i <= Lg; i++)
        if (pull >= i & 1) a = up[a][i];
    if (a == b) return a;
    for (int i = Lg; i >= 0; i--)
        if (up[a][i] != up[b][i])
            a = up[a][i], b = up[b][i];
    return up[a][0];
}
int jump(int x, int k) {
    for (int i = Lg; i >= 0; i--)
        if (k >= i & 1) x = up[x][i];
    return x;
}
int dist(int a, int b) {
    return dep[a] + dep[b] - 2 * dep[lca(a, b)];
}

```

2.2 Heavy Light Decomposition [0db4d5]

```

struct HLD {
    int n, cur = 0;
    vector<int> siz, top, dep, par, in, out, seq;
    vector<vector<int>> adj;
    HLD(int n) : n(n), siz(n),
        top(n), dep(n), par(n), in(n), out(n), seq(n), adj(n) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void work(int rt = 0) {
        top[rt] = rt;
        dep[rt] = 0;
        par[rt] = -1;
        dfs1(rt); dfs2(rt);
    }
    void dfs1(int u) {
        if (par[u] != -1) adj
            [u].erase(find(adj[u].begin(), adj[u].end(), par[u]));
        siz[u] = 1;
        for (auto &v : adj[u]) {
            par[v] = u, dep[v] = dep[u] + 1;
            dfs1(v);
            siz[u] += siz[v];
            if (siz[v] > siz[adj[u][0]]) {
                swap(v, adj[u][0]);
            } // 讓 adj[u][0] 是重子節點
        }
    }
    void dfs2(int u) {
        in[u] = cur++;
        seq[in[u]] = u; // dfn 對應的編號
        for (auto v : adj[u]) {
            top[v] = v == adj[u][0] ? top[u] : v;
            dfs2(v);
        }
        out[u] = cur;
    }
    int lca(int u, int v) {
        while (top[u] != top[v]) {

```

```

            if (dep[top[u]] > dep[top[v]]) {
                u = par[top[u]];
            } else {
                v = par[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    int dist(int u, int v) {
        return dep[u] + dep[v] - 2 * dep[lca(u, v)];
    }
    int jump(int u, int k) {
        if (dep[u] < k) return -1;
        int d = dep[u] - k;
        while (dep[top[u]] > d) u = par[top[u]];
        return seq[in[u] - dep[u] + d];
    } // c26ac4
    bool isAncestor(int u, int v) {
        return in[u] <= in[v] && in[v] < out[u];
    } // 2ca857
    int rootedParent(int rt, int v) {
        if (rt == v) return rt;
        if (!isAncestor(v, rt)) return par[v];
        auto it = upper_bound(adj[v].begin(), adj[v].end(), rt,
            [&](int x, int y) {
                return in[x] < in[y];
            }) - 1;
        return *it;
    } // c8db9f
    int rootedSize(int rt, int v) {
        if (rt == v) return n;
        if (!isAncestor(v, rt)) return siz[v];
        return n - siz[rootedParent(rt, v)];
    } // d7dea9
    int rootedLca(int rt, int a, int b) {
        return lca(rt, a) ^ lca(a, b) ^ lca(b, rt);
    } // da7f79
};

```

2.3 Centroid Decomposition [7ef37f]

```

vector<bool> vis(n);
vector<int> siz(n), par(n, -1);
auto findSize = [&](auto &&self, int u, int p) -> int {
    siz[u] = 1;
    for (int v : g[u]) {
        if (v == p || vis[v]) continue;
        siz[u] += self(self, v, u);
    }
    return siz[u];
};
auto findCen = [&](auto &&self, int u, int p, int sz) -> int {
    for (int v : g[u]) {
        if (v == p || vis[v]) continue;
        if (siz[v] * 2 > sz) return self(self, v, u, sz);
    }
    return u;
};
auto build = [&](auto &&self, int u, int p) -> void {
    findSize(self, u, p);
    int c = findCen(self, u, p, -1, siz[u]);
    vis[c] = true, par[c] = p;
    for (int v : g[c]) if (!vis[v]) self(self, v, c);
}; build(build, 0, -1);

```

2.4 SCC [414637]

```

struct SCC {
    int n, cur = 0, cnt = 0;
    vector<vector<int>> adj;
    vector<int> stk, dfn, low, bel;
    SCC(int n) : n(n), adj(n), dfn(n, -1), low(n), bel(n, -1) {}
    void addEdge(int u, int v) { adj[u].push_back(v); }
    void dfs(int x) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);
        for (auto y : adj[x]) {
            if (dfn[y] == -1) {
                dfs(y);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1) {
                low[x] = min(low[x], dfn[y]);
            }
        }
        if (dfn[x] == low[x]) {
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }
    vector<int> work() {
        for (int i = 0; i < n; i++)
            if (dfn[i] == -1) dfs(i);
        return bel;
    } // 1ebd5a
    struct Graph {
        int n;
        vector<pair<int, int>> edges;
    }
};

```

```

    vector<int> siz, cnte;
};
Graph compress() {
    Graph g;
    g.n = cnt;
    g.siz.resize(cnt), g.cnte.resize(cnt);
    for (int i = 0; i < n; i++) {
        g.siz[bel[i]]++;
        for (auto j : adj[i]) {
            if (bel[i] != bel[j]) {
                g.edges.emplace_back(bel[i], bel[j]);
            } else {
                g.cnte[bel[i]]++;
            }
        }
    }
    return g;
};

```

2.5 2-SAT [093413]

```

struct TwoSat : SCC {
    vector<int> ans;
    TwoSat(int n) : SCC(2 * n), ans(n) {}
    void addClause(int u, bool a, int v, bool b) {
        addEdge(2 * u + !a, 2 * v + b);
        addEdge(2 * v + !b, 2 * u + a);
    }
    void ifThen(int u, bool a, int v, bool b) {
        // 必取 A: not A -> A
        addEdge(2 * u + a, 2 * v + b);
    }
    bool work() {
        SCC::work(); // bel[!a] > bel[a] => !a -> a
        for (int i = 0; i < n / 2; i++) {
            if (bel[2 * i] == bel[2 * i + 1]) return false;
            ans[i] = bel[2 * i] > bel[2 * i + 1];
        }
        return true;
    }
};

```

2.6 EBCC [a84a20]

```

struct EBCC { // CF/contest/1986/pF
    int n, cur = 0, cnt = 0;
    vector<vector<int>> adj;
    vector<int> stk, dfn, low, bel;
    vector<pair<int, int>> bridges; // 關鍵邊
    EBCC(int n) : n(n), adj(n), low(n), dfn(n, -1), bel(n, -1) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int x, int p) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);
        for (auto y : adj[x]) {
            if (y == p) continue;
            if (dfn[y] == -1) {
                dfs(y, x);
                low[x] = min(low[x], low[y]);
                if (low[y] > dfn[x]) {
                    bridges.emplace_back(x, y);
                }
            } else if (bel[y] == -1) {
                low[x] = min(low[x], dfn[y]);
            }
        }
        if (dfn[x] == low[x]) {
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }
    vector<int> work() { // not connected
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) dfs(i, -1);
        }
        return bel;
    } // e0899d
    struct Graph {
        int n;
        vector<pair<int, int>> edges;
        vector<int> siz, cnte;
    };
    Graph compress() {
        Graph g; g.n = cnt;
        g.siz.resize(cnt); g.cnte.resize(cnt);
        for (int i = 0; i < n; i++) {
            g.siz[bel[i]]++;
            for (auto j : adj[i]) {
                if (bel[i] < bel[j]) {
                    g.edges.emplace_back(bel[i], bel[j]);
                } else if (bel[i] == bel[j] && i < j) {
                    g.cnte[bel[i]]++;
                }
            }
        }
    }
};

```

```

    }
    return g;
};

```

2.7 VBCC [30c4d8]

```

struct VBCC {
    int n, cur = 0, cnt = 0;
    vector<vector<int>> adj, bcc;
    vector<int> dfn, low, stk;
    vector<bool> ap;
    VBCC(int n) : n(n), adj(n), dfn(n, 0), low(n, 0), ap(n) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int x, int p = -1) {
        int ch = 0;
        dfn[x] = low[x] = ++cur;
        stk.push_back(x);
        for (int y : adj[x]) {
            if (!dfn[y]) {
                dfs(y, x);
                ch++;
                low[x] = min(low[x], low[y]);
                if (dfn[x] <= low[y]) {
                    ap[x] = true;
                    bcc.emplace_back();
                    int v;
                    do {
                        v = stk.back();
                        stk.pop_back();
                        bcc.back().push_back(v);
                    } while (v != y);
                    bcc.back().push_back(x);
                    cnt++;
                }
            } else if (dfn[y] < dfn[x] && y != p) {
                low[x] = min(low[x], dfn[y]);
            }
        }
        if (p == -1 && ch < 2) ap[x] = false;
    }
    void work() {
        for (int i = 0; i < n; i++) {
            if (!dfn[i]) {
                if (adj[i].empty()) {
                    bcc.emplace_back(1, i);
                    cnt++;
                } else dfs(i);
            } // 674cc5
        }
        struct Graph {
            int n;
            vector<pair<int, int>> edges;
            vector<int> bel, siz, cnte;
        };
        Graph compress() {
            Graph g;
            g.bel.assign(n, -1);
            g.siz.assign(cnt, 0);
            g.cnte.assign(cnt, 0);
            g.n = cnt;
            for (int i = 0; i < n; i++) {
                if (ap[i]) {
                    g.bel[i] = g.n++;
                    g.siz.push_back(1);
                    g.cnte.push_back(0);
                }
            }
            vector<bool> in(n);
            for (int i = 0; i < cnt; i++) {
                for (int u : bcc[i]) in[u] = true;
                int cnte = 0;
                for (int u : bcc[i]) {
                    if (ap[u]) {
                        g.edges.emplace_back(g.bel[u], i);
                    } else {
                        g.bel[u] = i, g.siz[i]++;
                    }
                }
                for (int v : adj[u]) if (in[v]) cnte++;
            }
            g.cnte[i] = cnte / 2;
            for (int u : bcc[i]) in[u] = false;
        }
        return g;
    }
};

```

2.8 Functional Graph [060d10]

```

const int N = 2E5;
const int Lg = __lg(N); // __lg(max(n, qi)), [0, Lg]
int cht[N][Lg];
struct FunctionalGraph {
    int n, cnt = 0;
    vector<int> g, bel, id, cycsz, in, top, hei;
    FunctionalGraph(const vector<int> &g) : n(g.size()) {
        g(g), bel(n, -1), id(n), in(n), top(n, -1), hei(n) {}
        for (int i = 0; i < n; i++) {
            cht[i][0] = g[i], in[g[i]]++;
        }
        for (int i = 1; i <= Lg; i++) {
            for (int u = 0; u < n; u++) {

```

```

    int nxt = cht[u][i - 1];
    cht[u][i] = cht[nxt][i - 1];
}
for (int i = 0; i < n; i++)
    if (in[i] == 0) label(i);
for (int i = 0; i < n; i++)
    if (top[i] == -1) label(i);
void label(int u) {
    vector<int> p; int cur = u;
    while (top[cur] == -1) {
        top[cur] = u;
        p.push_back(cur);
        cur = g[cur];
    }
    auto s = find(p.begin(), p.end(), cur);
    vector<int> cyc(s, p.end());
    p.erase(s, p.end()); p.push_back(cur);
    for (int i = 0; i < (int)cyc.size(); i++)
        bel[cyc[i]] = cnt, id[cyc[i]] = i, hei[cyc[i]] = cyc.size();
    if (!cyc.empty())
        ++cnt, cycsz.push_back(cyc.size());
    for (int i = p.size() - 1; i > 0; i--)
        id[p[i] - 1] = id[p[i]] - 1, hei[p[i] - 1] = hei[p[i]] + 1;
}
int jump(int u, int k) {
    for (int b = 0; k > 0; b++) {
        if (k & 1) u = cht[u][b];
        k >>= 1;
    }
    return u;
}
};

```

2.9 Euler [13ce48]

```

// 1. 無向圖是歐拉圖：
// 非零度頂點是連通的
// 頂點的度數都是偶數

// 2. 無向圖是半歐拉圖(有路沒有環)：
// 非零度頂點是連通的
// 恰有 2 個奇度頂點

// 3. 有向圖是歐拉圖：
// 非零度頂點是強連通的
// 每個頂點的入度和出度相等

// 4. 有向圖是半歐拉圖(有路沒有環)：
// 非零度頂點是弱連通的
// 至多一個頂點的出度與入度之差為 1
// 至多一個頂點的入度與出度之差為 1
// 其他頂點的入度和出度相等
vector<int> ans;
[&](this auto &&self, int u) -> void {
    while (g[u].size()) {
        int v = *g[u].begin();
        g[u].erase(v);
        self(v);
    }
    ans.push_back(u);
} (0); reverse(ans.begin(), ans.end());

```

2.10 Virtual Tree [b2b0eb]

```

// 多次詢問給某些關鍵點，虛樹可達成快速樹 DP (前處理每個點)
// 例如這題是有權樹，給一些關鍵點，求跟 vertex 1 隔開的最小成本
// 前處理 root 到所有點的最小邊權
void solve(int n, vector<vector<pair<int, int>>> &g, int q) {
    build(n, g); // build LCA
    vector<int> dis(n, 1E9); // root 到各點的最小邊權
    [&](this auto &&self, int x, int p) -> void {
        for (auto [y, w] : g[x]) {
            if (y == p) continue;
            dis[y] = min(w, dis[x]);
            self(y, x);
        }
    } (0, -1);
    vector<vector<int>> vt(n);
    vector<int> isKey(n);
    vector<ll> dp(n);
    while (q--) {
        int m; cin >> m;
        vector<int> key(m);
        for (int i = 0; i < m; i++) {
            cin >> key[i];
            isKey[--key[i]] = 1;
        }
        key.push_back(0); // 固定 0 為 root，看題目需求
        sort(key.begin(), key.end(), [&](int a, int b) {
            return dfn[a] < dfn[b];
        }); // 要 sort 再 insert
        vector<int> stk; // 維護一條虛樹鏈
        for (int k : key) {
            if (stk.empty()) {
                stk.push_back(k);
            }

```

```

            continue;
        }
        int l = lca(stk.back(), k);
        if (l == stk.back()) { // 直接一條鏈
            stk.push_back(k);
            continue;
        } // 次頂是 lca 的子孫，直接跟最頂連邊
        while (stk.size() > 1 && dfn[stk[stk.size() - 2]] > dfn[l]) {
            vt[stk[stk.size() - 2]].push_back(stk.back());
            stk.pop_back();
        } // 保證 back 是 lca 的子孫，且次頂不是 lca 的子孫
        vt[l].push_back(stk.back());
        if (stk.size() < 2 || stk[stk.size() - 2] != l) stk.back() = l;
        else stk.pop_back(); // lca 的子孫已經用不到了，直接換成 lca
        stk.push_back(k);
    }
    while (stk.size() > 1) {
        vt[stk[stk.size() - 2]].push_back(stk.back());
        stk.pop_back();
    }
    int rt = stk[0]; stk.clear();
    [&](this auto &&self, int x) -> void {
        for (auto y : vt[x]) {
            self(y);
            if (isKey[y]) { // 直接砍了
                dp[x] += dis[y];
            } else { // 不砍 or 砍
                dp[x] += min<ll>(dp[y], dis[y]);
            } // 記得 reset
            isKey[y] = dp[y] = 0;
        }
        vt[x].clear(); // 記得 reset
    } (rt);
    cout << dp[rt] << "\n";
    isKey[rt] = 0;
    dp[rt] = 0; // 最後 reset root
}
}

```

2.11 Dominator Tree [a54c17]

```

// dom
// 存起點到達此點的必經的上個節點(起點 = 自己)，無法到達 = -1
struct DominatorTree {
    int n, id = 0;
    vector<vector<int>> adj, radj, bucket;
    vector<int> sdom, dom, vis, rev, pa, rt, mn, res;
    DominatorTree(int n) : n(n), adj(n), radj(n), bucket(n), sdom(n), dom(n),
        -1, vis(n, -1), rev(n), pa(n), rt(n), mn(n), res(n) {}
    void addEdge(int u, int v) { adj[u].push_back(v); }
    int query(int v, int x) {
        if (rt[v] == v) return x ? -1 : v;
        int p = query(rt[v], 1);
        if (p == -1) return x ? rt[v] : mn[v];
        if (sdom[mn[v]] > sdom[mn[rt[v]]]) mn[v] = mn[rt[v]];
        rt[v] = p;
        return x ? p : mn[v];
    }
    void dfs(int v) {
        vis[v] = id, rev[id] = v;
        rt[id] = mn[id] = sdom[id] = id, id++;
        for (int u : adj[v]) {
            if (vis[u] == -1) dfs(u), pa[vis[u]] = vis[v];
            radj[vis[u]].push_back(vis[v]);
        }
    }
    vector<int> build(int s) {
        dfs(s);
        for (int i = id - 1; i >= 0; i--) {
            for (int u : radj[i])
                sdom[i] = min(sdom[i], sdom[query(u, 0)]);
            if (i) bucket[sdom[i]].push_back(i);
            for (int u : bucket[i]) {
                int p = query(u, 0);
                dom[u] = sdom[p] == i ? i : p;
            }
            if (i) rt[i] = pa[i];
        }
        res.assign(n, -1);
        for (int i = 1; i < id; i++)
            if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
        for (int i = 1; i < id; i++) res[rev[i]] = rev[dom[i]];
        res[s] = s;
        for (int i = 0; i < n; i++) dom[i] = res[i];
        return dom;
    }
};

```

3 Data Structure

3.1 Fenwick [d41d8c]

```

template<class T> struct Fenwick {
    int n; vector<T> a;
    Fenwick(int n) : n(n), a(n) {}

```

```

void add(int x, const T &v) {
    for (x++; x <= n; x += x & -x) a[x - 1] = a[x - 1] + v;
}
T sum(int x) {
    T v{};
    for (; x > 0; x -= x & -x) v = v + a[x - 1];
    return v;
}
T rangeSum(int l, int r) { return sum(r) - sum(l); } /*
int select(const T &k, int start = 0) {
    // 找到最小的 x, 使得 sum(x + 1) - sum(start) > k
    // prefix sum 要有單調性
    int x = 0;
    T cur = -sum(start);
    for (int i = 1 << __lg(n); i > 0; i /= 2)
        if (x + i <= n && cur + a[x + i - 1] <= k)
            x += i, cur = cur + a[x - 1];
    return x;
} */ // 8e1de2
}; // 9083e4
template<class T> struct Fenwick2D {
    int n, m;
    vector<vector<T>> a;
    Fenwick2D(int n, int m) : n(n), m(m), a(n, vector<T>(m)) {}
    void add(int x, int y, const T &v) {
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j)
                a[i - 1][j - 1] = a[i - 1][j - 1] + v;
    }
    T sum(int x, int y) {
        T v{};
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j)
                v = v + a[i - 1][j - 1];
        return v;
    }
    T rangeSum(int x1, int y1, int x2, int y2) {
        return sum(x2, y2) - sum(x1, y2) - sum(x2, y1) + sum(x1, y1);
    }
}; // a71e39
template<class T> struct RangeFenwick {
    int n;
    vector<T> d, di;
    RangeFenwick(int n) : n(n), d(n), di(n) {}
    void add(int x, const T &v) {
        T vi = v * (x + 1);
        for (int i = x + 1; i <= n; i += i & -i)
            d[i - 1] = d[i - 1] + v, di[i - 1] = di[i - 1] + vi;
    }
    void rangeAdd(int l, int r, const T &v) { add(l, v), add(r, -v); }
    T sum(int x) {
        T v{};
        for (int i = x; i > 0; i -= i & -i)
            v = v + T(x + 1) * d[i - 1] - di[i - 1];
        return v;
    }
    T rangeSum(int l, int r) { return sum(r) - sum(l); } /*
int select(const T &k, int start = 0) {
    int x = 0;
    T cur{}, curi{}, sub = sum(start);
    for (int i = 1 << __lg(n); i; i /= 2) {
        if (x + i <= n && T(x + i + 1) * (cur +
            d[x + i - 1]) - (curi + di[x + i - 1]) - sub <= k) {
            x += i;
            cur = cur + d[x - 1], curi = curi + di[x - 1];
        }
    }
    return x;
} */ // e93ec7
}; // 03784e
template<class T> struct RangeFenwick2D {
    int n, m;
    vector<vector<T>> d, di, dj, dij;
    RangeFenwick2D(int n, int m) : n(n), m(m), dij(n, vector<T>(m)) {
        d = di = dj = dij;
    }
    void add(int x, int y, const T &v) {
        T vi = v * (x + 1), vj = v * (y + 1);
        T vij = v * (x + 1) * (y + 1);
        for (int i = x + 1; i <= n; i += i & -i)
            for (int j = y + 1; j <= m; j += j & -j) {
                d[i - 1][j - 1] = d[i - 1][j - 1] + v;
                di[i - 1][j - 1] = di[i - 1][j - 1] + vi;
                dj[i - 1][j - 1] = dj[i - 1][j - 1] + vj;
                dij[i - 1][j - 1] = dij[i - 1][j - 1] + vij;
            }
    }
    void rangeAdd(int x1, int y1, int x2, int y2, const T &v) {
        add(x1, y1, v), add(x2, y2, v);
        add(x1, y2, -v), add(x2, y1, -v);
    }
    T sum(int x, int y) {
        T v{};
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j)
                v = v + (x + 1) * (y + 1) * d[i - 1][j - 1]
                    - (y + 1) * di[i - 1][j - 1]
                    - (x + 1) * dj[i - 1][j - 1]
                    + dij[i - 1][j - 1];
        return v;
    }
};

```

```

}
T rangeSum(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x1, y2) - sum(x2, y1) + sum(x1, y1);
} // 60ea46

```

3.2 Lazy Segment Tree [d41d8c]

```

template<class Info, class Tag> struct LazySegmentTree {
    int n;
    vector<Info> info;
    vector<Tag> tag;
    LazySegmentTree(int n) : n(n), info(4 << __lg(n)), tag(4 << __lg(n)) {}
    LazySegmentTree(const vector<Info> &v) : n(v.size()), info(4 << __lg(n)), tag(4 << __lg(n)) {}
    auto build = [&](auto &&self, int p, int l, int r) {
        if (r - l == 1) {
            info[p] = v[l];
            return;
        }
        int m = (l + r) / 2;
        self(self, 2 * p, l, m);
        self(self, 2 * p + 1, m, r);
        pull(p);
    };
    build(build, 1, 0, n);
    void pull(int p) { info[p] = info[2 * p] + info[2 * p + 1]; }
    void apply(int p, int l, int r, const Tag &t) {
        info[p].apply(t, l, r);
        tag[p].apply(t);
    }
    void push(int p, int l, int r) {
        int m = (l + r) / 2;
        apply(2 * p, l, m, tag[p]);
        apply(2 * p + 1, m, r, tag[p]);
        tag[p] = Tag();
    }
    void modify(int x, const Info &i) { modify(1, 0, n, x, i); }
    void modify(int p, int l, int r, int x, const Info &i) {
        if (r - l == 1) {
            info[p] = i; // add use +=
            return;
        }
        int m = (l + r) / 2;
        push(p, l, r);
        if (x < m) modify(2 * p, l, m, x, i);
        else modify(2 * p + 1, m, r, x, i);
        pull(p);
    }
    Info query(int ql, int qr) { return query(1, 0, n, ql, qr); }
    Info query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || ql >= r) return Info();
        if (ql <= l && r <= qr) return info[p];
        int m = (l + r) / 2;
        push(p, l, r);
        return query(2 * p, l, m, ql, qr) + query(2 * p + 1, m, r, ql, qr);
    }
    void rangeApply(int ql, int qr, const Tag &t) { rangeApply(1, 0, n, ql, qr, t); }
    void rangeApply(int p, int l, int r, int ql, int qr, const Tag &t) {
        if (qr <= l || ql >= r) return;
        if (ql <= l && r <= qr) {
            apply(p, l, r, t.offset(l - ql)); // tag 可位移 (l - ql)
            return;
        }
        int m = (l + r) / 2;
        push(p, l, r);
        rangeApply(2 * p, l, m, ql, qr, t);
        rangeApply(2 * p + 1, m, r, ql, qr, t);
        pull(p);
    } /*
    template<class F> int findFirst(int ql, int qr, F &&f) {
        return findFirst(1, 0, n, ql, qr, f);
    } // 若要找 last, 先右子樹遍迴即可
    template<class F> int findFirst(int p, int l, int r, int ql, int qr, F &&f) {
        if (qr <= l || ql >= r || !f(info[p])) return -1;
        if (r - l == 1) return l;
        int m = (l + r) / 2;
        push(p, l, r);
        int res = findFirst(2 * p, l, m, ql, qr, f);
        if (res == -1) res = findFirst(2 * p + 1, m, r, ql, qr, f);
        return res;
    } */
};
struct Tag { // 有些 Tag 不用 push 例如 sweepLine
    void apply(const Tag &t) {}
    Tag offset(int d) const { return *this; }
};
struct Info {
    void apply(const Tag &t, int l, int r) {} /*
    Info &operator=(const Info &i) { // partial assignment
        return *this;
    }
    Info &operator+=(const Info &i) { // leaf add
        return *this = *this + i;
    }
};

```

```

} */
};
Info operator+(const Info &a, const Info &b) {
    Info c;
    return c;
}

```

3.3 Dynamic Segment Tree [d41d8c]

```

template<class Info> struct SegmentTreeDynamic {
    struct Node {
        Info info = Info();
        int lc = 0, rc = 0;
    };
    int n;
    vector<Node> nd;
    SegmentTreeDynamic(int n) : n(n), nd(1) {}
    int generate() {
        nd.emplace_back();
        return nd.size() - 1;
    }
    void pull(int p) {
        nd[p].info = nd[nd[p].lc].info + nd[nd[p].rc].info;
    }
    int modify(int x, const Info &i, int rt = 0) { return modify(rt, 0, n, x, i); }
    int modify(int p, int l, int r, int x, const Info &i) {
        if (p == 0) p = generate();
        if (r - l == 1) {
            nd[p].info = i; // add use +=
            return p;
        }
        int m = (l + r) / 2;
        if (x < m) nd[p].lc = modify(nd[p].lc, l, m, x, i);
        else nd[p].rc = modify(nd[p].rc, m, r, x, i);
        pull(p);
        return p;
    }
    int merge(int p, int q) { return merge(p, q, 0, n); }
    int merge(int p, int q, int l, int r) {
        if (p == 0 || q == 0) return p + q;
        if (r - l == 1) {
            nd[p].info += nd[q].info; // leaf add
            return p;
        }
        int m = (l + r) / 2;
        nd[p].lc = merge(nd[p].lc, nd[q].lc, l, m);
        nd[p].rc = merge(nd[p].rc, nd[q].rc, m, r);
        pull(p);
        return p;
    }
    Info query(int ql, int qr, int rt = 0) { return query(rt, 0, n, ql, qr); }
    Info query(int p, int l, int r, int ql, int qr) {
        if (l >= qr || r <= ql || p == 0) return Info();
        if (ql <= l && r <= qr) return nd[p].info;
        int m = (l + r) / 2;
        return query(nd[p].lc, l, m, ql, qr) + query(nd[p].rc, m, r, ql, qr);
    }
    /*
    template<class F> int findFirst(int ql, int qr, F &&f, int rt = 0) {
        return findFirst(rt, 0, n, ql, qr, f);
    }
    template<class F> int findFirst(int p, int l, int r, int ql, int qr, F &&f) {
        if (l >= qr || r <= ql || p == 0 || !f(nd[p].info)) return -1;
        if (r - l == 1) return l;
        int m = (l + r) / 2;
        int res = findFirst(nd[p].lc, l, m, ql, qr, f);
        if (res != -1) return res;
        return findFirst(nd[p].rc, m, r, ql, qr, f);
    }
    */
};

```

3.4 Persistent Segment Tree [d41d8c]

```

template<class Info> struct PST {
    struct Node {
        Info info = Info();
        int lc = 0, rc = 0;
    };
    int n;
    vector<Node> nd;
    vector<int> rt;
    PST(int n) : n(n), nd(1), rt(1) {}
    PST(const vector<Info> &v) : n(v.size()), nd(1) {
        auto build = [&](auto &&self, int l, int r) -> int {
            int p = copy(0);
            if (r - l == 1) {
                nd[p].info = v[l];
                return p;
            }
            int m = (l + r) / 2;
            int lc = self(self, l, m);
            int rc = self(self, m, r);
            nd[p].lc = lc;
            nd[p].rc = rc;
            pull(nd[p]);
            return p;
        };
        rt.push_back(build(build, 0, n));
    }
};

```

```

}
void pull
(Node &p) { p.info = nd[p.lc].info + nd[p.rc].info; }
int copy(int p) {
    nd.push_back(nd[p]);
    return nd.size() - 1;
}
void modify(int x, const Info &i, int ver = 0) {
    assert(rt.size() > ver);
    rt[ver] = modify(rt[ver], 0, n, x, i);
}
int modify(int p, int l, int r, int x, const Info &i) {
    p = copy(p);
    if (r - l == 1) {
        nd[p].info = i;
        return p;
    }
    int m = (l + r) / 2;
    if (x < m) nd[p].lc = modify(nd[p].lc, l, m, x, i);
    else nd[p].rc = modify(nd[p].rc, m, r, x, i);
    pull(nd[p]);
    return p;
}
Info query(int ql, int qr, int ver = 0) {
    return query(rt[ver], 0, n, ql, qr);
}
Info query(int p, int l, int r, int ql, int qr) {
    if (l >= qr || r <= ql || p == 0) return Info();
    if (ql <= l && r <= qr) return nd[p].info;
    int m = (l + r) / 2;
    return query(nd[p].lc, l, m, ql, qr) + query(nd[p].rc, m, r, ql, qr);
}
int createVersion(int ver) {
    rt.push_back(rt[ver]);
    return rt.size() - 1;
}
};

```

3.5 Kth-element [d41d8c]

```

template<class T> struct StaticKth : PST<int> {
    int dct(T x) {
        return lower_bound(s.begin(), s.end(), x) - s.begin();
    }
    vector<T> a, s; // array, sorted
    StaticKth(const vector<T> &a, const vector<T> &s) : a(a), s(s), PST<int>(s.size()) {
        assert(is_sorted(s.begin(), s.end()));
        for (int i = 0; i < a.size(); i++) {
            createVersion(i);
            add(dct(a[i]), 1, i + 1);
        }
    }
    int work(int a, int b, int l, int r, int k) {
        if (r - l == 1) return l;
        int x = nd[nd[b].lc].info - nd[nd[a].lc].info;
        int m = (l + r) / 2;
        if (x >= k) return work(nd[a].lc, nd[b].lc, l, m, k);
        else return work(nd[a].rc, nd[b].rc, m, r, k - x);
    }
    int work(int l, int r, int k) { // [l, r), k > 0
        return s[work(rt[l], rt[r], 0, n, k)];
    }
}; // Dynamic -> Fenwick 包動態開點值域線段樹

```

3.6 Link Cut Tree [d41d8c]

```

// 有用到 pathApply 才需要 apply 有關的
// const int Mod = 51061;
struct Tag {
    // ll add = 0, mul = 1;
    void apply(const Tag &t) {
        // mul = mul * t.mul % Mod;
        // add = (add * t.mul % Mod + t.add) % Mod;
    }
};
struct Info {
    // int siz = 0; // 實鍵的長度
    // ll val = 0, sum = 0;
    void apply(const Tag &t) {
        // val = (val * t.mul % Mod + t.add) % Mod;
        // sum = (sum * t.mul % Mod + t.add * siz % Mod) % Mod;
    }
    void pull(const Info &l, const Info &r) {
        // siz = 1 + l.siz + r.siz;
        // sum = (l.sum + r.sum + val) % Mod;
    }
};
struct LinkCutTree { // 1-based
    vector<Info> info;
    vector<Tag> tag;
    vector<array<int, 2>> ch;
    vector<int> p, rev;
    LinkCutTree(int n) : info(n + 1), tag(n + 1), ch(n + 1), p(n + 1), rev(n + 1) {}
    bool isrt(int x) {
        return ch[p[x]][0] != x && ch[p[x]][1] != x;
    }
    int pos(int x) { // x 是其 par 的左/右
        return ch[p[x]][1] == x;
    }
};

```

```

void applyRev(int x) {
    if (!x) return;
    swap(ch[x][0], ch[x][1]);
    rev[x] ^= 1;
}
void apply(int x, const Tag &t) {
    if (!x) return;
    info[x].apply(t);
    tag[x].apply(t);
}
void push(int x) {
    if (rev[x]) {
        applyRev(ch[x][0]);
        applyRev(ch[x][1]);
        rev[x] = 0;
    }
    apply(ch[x][0], tag[x]);
    apply(ch[x][1], tag[x]);
    tag[x] = Tag();
}
void pull(int x) {
    if (!x) return;
    info[x].pull(info[ch[x][0]], info[ch[x][1]]);
}
void pushAll(int x) {
    if (!isrt(x)) pushAll(p[x]);
    push(x);
}
void rotate(int x) { // x 與其 par 交換位置
    int f = p[x], r = pos(x);
    ch[f][r] = ch[x][!r];
    if (ch[x][!r]) p[ch[x][!r]] = f;
    p[x] = p[f];
    if (!isrt(f)) ch[p[f]][pos(f)] = x;
    ch[x][!r] = f, p[f] = x;
    pull(f), pull(x);
}
void splay(int x) { // x 旋轉到當前的根
    pushAll(x);
    for (int f = p[x]; f = p[x], !isrt(x); rotate(x))
        if (!isrt(f)) rotate(pos(x) == pos(f) ? f : x);
}
// access(x), access(y) 可以回傳 LCA
int access(int x) { // 根到 x 換成實鍵
    int c;
    for (c = 0; x; c = x, x = p[x]) {
        splay(x);
        ch[x][1] = c;
        pull(x);
    }
    return c;
}
void makeRoot(int x) { // x 變成所在樹的根
    access(x), splay(x), applyRev(x);
}
int findRoot(int x) {
    access(x), splay(x);
    while (ch[x][0]) x = ch[x][0];
    splay(x); return x;
}
void split(int rt, int x) { // 以 rt 為根, x 為子樹根
    makeRoot(x), access(rt), splay(rt);
}
void link(int rt, int x) { // 以 rt 為根, x 為子樹根
    makeRoot(rt);
    access(x), splay(x);
    p[rt] = x;
    pull(x);
}
void cut(int rt, int x) { // 以 rt 為根, x 為子樹根
    split(rt, x);
    ch[rt][0] = p[x] = 0;
    pull(rt);
}
bool connected(int x, int y) {
    return findRoot(x) == findRoot(y);
}
bool neighbor(int x, int y) {
    if (!connected(x, y)) return false;
    split(x, y);
    return info[x].siz == 2; // psiz
}
void modify(int x, const Info &i) {
    access(x), splay(x);
    info[x] = i, pull(x);
}
void pathApply(int x, int y, const Tag &t) {
    assert(connected(x, y));
    split(x, y), apply(x, t);
}
Info pathQuery(int x, int y) {
    assert(connected(x, y));
    split(x, y); return info[x];
}
};

```

3.7 LCT Extension [d41d8c]

```

struct Info {
    int siz = 0, vsiz
        = 0, psiz = 0; // 真實子樹大小, 虛子樹大小, 實鍵長度
};

```

```

ll val = 0, sum = 0, vsum = 0, psum = 0;
void apply(const Tag &t) {
    val = (val * t.mul % Mod + t.add) % Mod;
    sum = (sum - psum + Mod) % Mod;
    psum = (psum * t.mul % Mod + t.add * psiz % Mod) % Mod;
    sum = (sum + psum) % Mod;
}
void pull(const Info &l, const Info &r) {
    psiz = 1 + l.psiz + r.psiz;
    siz = 1 + l.siz + r.siz + vsiz;
    psum = (l.psum + r.psum + val) % Mod;
    sum = (l.sum + r.sum + val + vsum) % Mod;
}
void modify(const Info &i) { // 才不會蓋掉 vsum, vsiz
    val = i.val;
}
// 下面的 modify 也要改, 不能直接 =
void operator+=(const Info &i) { // 將實邊轉虛邊時加上資訊
    vsiz += i.siz;
    vsum = (vsum + i.sum) % Mod;
}
void operator-=(const Info &i) { // 將虛邊轉實邊時扣除資訊
    vsiz -= i.siz;
    vsum = (vsum - i.sum % Mod + Mod) % Mod;
}
Info asSubtree
    () const { // 當前節點作為 Virtual Subtree 根時的狀態
    Info res;
    res.siz = vsiz + 1;
    res.sum = (vsum + val) % Mod;
    return res;
}
};
struct LinkCutTree { // 加在註解後面
    int access(int x) { // ...splay(x);
        if (ch[x][1]) info[x] += info[ch[x][1]]; // 原右子樹轉虛邊
        if (c) info[x] -= info[c]; // 虛邊轉新右子樹
    }
    void link(int rt, int x) { // ...p[rt] = x;
        info[x] += info[rt]; // rt 成為 x 的虛子樹
    }
    Info subtreeQuery
        (int rt, int x) { // 以 rt 當 root, x 的 subtree
        assert(connected(rt, x));
        makeRoot(rt);
        access(x), splay(x);
        return info[x].asSubtree();
    }
    Info componentQuery(int x) {
        makeRoot(x);
        return info[x];
    }
};

```

3.8 Treap [d41d8c]

```

template<class Info, class Tag = bool>
struct Treap { // 0 -> initial root
    vector<Info> info;
    // vector<Tag> tag;
    vector<int> siz, par, rev, pri;
    vector<array<int, 2>> ch;
    Treap(int n) : info(n + 1), siz
        (n + 1), par(n + 1), rev(n + 1), pri(n + 1), ch(n + 1) {
        // tag.resize(n + 1);
        for (int i = 1; i <= n; i++) siz[i] = 1, pri[i] = rng();
    }
    /*
    void apply(int p, const Tag &t) {
        info[p].apply(siz[p], t);
        tag[p].apply(t);
    }
    */
    void push(int p) {
        if (rev[p]) {
            swap(ch[p][0], ch[p][1]);
            if (ch[p][0]) rev[ch[p][0]] ^= 1;
            if (ch[p][1]) rev[ch[p][1]] ^= 1;
            rev[p] = 0;
        }
        /*
        apply(ch[p][0], tag[p]);
        apply(ch[p][1], tag[p]);
        tag[p] = Tag(); */
    }
    void pull(int p) {
        siz[p] = 1 + siz[ch[p][0]] + siz[ch[p][1]];
        par[ch[p][0]] = par[ch[p][1]] = p;
        info[p].pull(info[ch[p][0]], info[ch[p][1]]);
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        push(a), push(b);
        if (pri[a] > pri[b]) {
            ch[a][1] = merge(ch[a][1], b);
            pull(a); return a;
        } else {
            ch[b][0] = merge(a, ch[b][0]);
            pull(b); return b;
        }
    }
    pair<int, int> split(int p, int k) {
        if (!p) return {0, 0};
        push(p);
    }
};

```

```

    if (siz[ch[p][0]] >= k) {
        auto [a, b] = split(ch[p][0], k);
        ch[p][0] = b, pull(p);
        return {a, p};
    } else {
        auto [a, b] = split(ch[p][1], k - siz[ch[p][0]] - 1);
        ch[p][1] = a, pull(p);
        return {p, b};
    }
}

void getArray(int p, vector<Info> &a) {
    if (!p) return;
    push(p);
    getArray(ch[p][0], a);
    a.push_back(info[p]);
    getArray(ch[p][1], a);
}

/*
int getPos(int rt, int p) { // get p's index in array
    int k = siz[ch[p][0]] + 1;
    for (; ; p = par[p]) {
        if (rev[p]) k = siz[p] + 1 - k;
        if (p == rt) return k;
        if (ch[par[p]][1] == p) k += siz[ch[par[p]][0]] + 1;
    }
}

template<class F> int findFirst(int p, F &pred) {
    if (!p) return 0;
    push(p);
    if (!pred(info[p])) return 0;
    int idx = findFirst(ch[p][0], pred);
    if (!idx)
        idx = 1 + siz[ch[p][0]] + findFirst(ch[p][1], pred);
    return idx;
}

*/
struct Tag {
    int setv; ll add;
    void apply(const Tag &t) {
        if (t.setv) {
            setv = t.setv;
            add = t.add;
        } else {
            add += t.add;
        }
    }
};

/*
struct Info {
    ll val, sum;
    void apply(int siz, const Tag &t) {
        if (t.setv) {
            val = t.setv;
            sum = 1LL * siz * t.setv;
        }
        val += t.add;
        sum += 1LL * siz * t.add;
    }
}

void pull(const Info &l, const Info &r) {
    // sum = val + l.sum + r.sum;
}
};

```

3.9 KDTree [d41d8c]

```

struct Info {
    static constexpr int DIM = 2;
    array<int, DIM> x, l, r;
    int v = 0, sum = 0;
    void pull(const Info &l, const Info &r) {
        sum = v + l.sum + r.sum;
    }
};

struct KDTree {
    static constexpr int DIM = Info::DIM;
    vector<Info> info;
    vector<int> rt, l, r, p;
    int n, lg;
    KDTree(int n) : n(n),
        lg(__lg(n)), info(1), rt(lg + 1), l(n + 1), r(n + 1) {}
    void pull(int p) {
        info[p].l = info[p].r = info[p].x;
        info[p].pull(info[l[p]], info[r[p]]);
        for (int ch : {l[p], r[p]}) {
            if (!ch) continue;
            for (int k = 0; k < DIM; k++) {
                info[p].l[k] = min(info[p].l[k], info[ch].l[k]);
                info[p].r[k] = max(info[p].r[k], info[ch].r[k]);
            }
        }
    }

    int rebuild(int l, int r, int dep = 0) {
        if (r == l) return 0;
        int m = (l + r) / 2;
        nth_element(p.begin() + l, p.begin() + m, p.begin() + r,
            [&](int x,
                int y) { return info[x].x[dep] < info[y].x[dep]; });
        int x = p[m];
        this->l[x] = rebuild(l, m, (dep + 1) % DIM);
        this->r[x] = rebuild(m + 1, r, (dep + 1) % DIM);
        pull(x);
        return x;
    }

    void append(int &x) {

```

```

        if (!x) return;
        p.push_back(x);
        append(l[x]);
        append(r[x]);
        x = 0;
    }

    void addNode(const Info &i) {
        p.assign(1, info.size());
        info.push_back(i);
        for (int j = 0; j++) {
            if (!rt[j]) {
                rt[j] = rebuild(0, p.size());
                break;
            } else {
                append(rt[j]);
            }
        }
    }

    Info query(int
        p, const array<int, DIM> &l, const array<int, DIM> &r) {
        if (!p) return Info();
        bool inside = true;
        for (int k = 0; k < DIM; k++) {
            inside &= (l[k] <= info[p].l[k] && info[p].r[k] <= r[k]);
        }
        if (inside) return info[p];
        for (int k = 0; k < DIM; k++) {
            if (info[p].r[k] < l[k] || r[k] < info[p].l[k]) {
                return Info();
            }
        }
        Info ans;
        inside = true;
        for (int k = 0; k < DIM; k++) {
            inside &= l[k] <= info[p].x[k] && info[p].x[k] <= r[k];
        }
        if (inside) ans = info[p];
        ans.pull(query(this->l[p], l, r), query(this->r[p], l, r));
        return ans;
    }

    Info query
        (const array<int, DIM> &l, const array<int, DIM> &r) {
        Info res;
        for (int i = 0; i <= lg; i++) {
            res.pull(res, query(rt[i], l, r));
        }
        return res;
    }
};

```

3.10 Sparse Table [d41d8c]

```

template<class T, class F = less<T>> struct RMQ { // [l, r)
    int n;
    F cmp = F();
    vector<vector<T>> g;
    RMQ() {}
    RMQ(const vector<T> &a, F cmp = F()) : cmp(cmp) {
        init(a);
    }
    void init(const vector<T> &a) {
        n = a.size();
        if (!n) return;
        int lg = __lg(n);
        g.resize(lg + 1);
        g[0] = a;
        for (int j = 1; j <= lg; j++) {
            g[j].resize(n - (1 << j) + 1);
            for (int i = 0; i <= n - (1 << j); i++)
                g[j][i] = min
                    (g[j - 1][i], g[j - 1][i + (1 << (j - 1))], cmp);
        }
    }
    T operator()(int l, int r) {
        assert(0 <= l && l < r && r <= n);
        int lg = __lg(r - l);
        return min(g[lg][l], g[lg][r - (1 << lg)], cmp);
    }
};

```

3.11 Mo [d41d8c]

```

struct Query { int id, l, r; };
void mo(vector<Query> &q) {
    int blk = sqrt(q.size());
    sort(q
        .begin(), q.end(), [&](const Query &a, const Query &b) {
            int x = a.l / blk, y = b.l / blk;
            return x == y ? a.r < b.r : x < y;
        });
    int nl = 0, nr = -1;
    for (auto [id, l, r] : q) {
        while (nr < r) nr++, addR();
        while (l < nl) nl--, addL();
        while (r < nr) delR(), nr--;
        while (nl < l) delL(), nl++;
    }
}

```

3.12 Rollback DSU [d41d8c]

```

struct RollbackDSU {
    int n;
    vector<int> f, siz, stk;
    RollbackDSU(int n) : n(n), f(n), siz(n, 1) {
        iota(f.begin(), f.end(), 0);
    }
    int find(int x) { return x == f[x] ? x : find(f[x]); }
    bool same(int x, int y) { return find(x) == find(y); }
    bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        siz[x] += siz[y];
        f[y] = x;
        n--;
        stk.push_back(y);
        return true;
    }
    void undo(int x) {
        while (stk.size() > x) {
            int y = stk.back();
            stk.pop_back();
            n++;
            siz[f[y]] -= siz[y];
            f[y] = y;
        }
    }
    int size(int x) { return siz[find(x)]; }
};

```

4 Flow Matching

4.1 Dinic [d41d8c]

```

template<class T> struct Dinic {
    // augment time: O(VE), O(E) for unit capacity,
    // augment number: min(O(F), O(V))
    // min(O(E^0.5), O(V^2/3)) for unit capacity
    // O(V^0.5) for deg_in(u) or deg_out(u) <= 1
    // so bipartite matching: O(EV^0.5)
    struct Edge {
        int to;
        T f, cap; // 流量跟容量
    };
    int n, m, s, t;
    const T INF_FLOW = numeric_limits<T>::max() / 2;
    vector<vector<int>> g;
    vector<Edge> e;
    vector<int> h, cur;
    Dinic(int n) : n(n), m(0), g(n), h(n), cur(n) {}
    void addEdge(int u, int v, T cap) {
        e.push_back({v, 0, cap});
        e.push_back({u, 0, 0});
        g[u].push_back(m++);
        g[v].push_back(m++);
    }
    bool bfs() {
        fill(h.begin(), h.end(), -1);
        h[s] = 0; queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u]) {
                auto [v, f, cap] = e[id];
                if (f != cap && h[v] == -1) {
                    h[v] = h[u] + 1;
                    if (v == t) return true;
                    q.push(v);
                }
            }
        }
        return false;
    }
    T dfs(int u, T flow) {
        if (flow == 0) return 0;
        if (u == t) return flow;
        for (int &i = cur[u]; i < g[u].size(); i++) {
            int j = g[u][i];
            auto [v, f, cap] = e[j];
            if (f == cap || h[u] + 1 != h[v]) continue;
            T mn = dfs(v, min(flow, cap - f));
            if (mn > 0) {
                e[j].f += mn;
                e[j ^ 1].f -= mn;
                return mn;
            }
        }
        return 0;
    }
    T work(int s_, int t_) {
        s = s_; t = t_; T f = 0;
        while (bfs()) {
            fill(cur.begin(), cur.end(), 0);
            while (true) {
                T res = dfs(s, INF_FLOW);
                if (res == 0) break;
                f += res;
            }
        }
        return f;
    }
    /*
    void reuse(int n_) { // 走殘留網路, res += f

```

```

        while (n < n_) {
            g.emplace_back();
            h.emplace_back();
            cur.emplace_back();
            n += 1;
        }
    } // 885d65
}; // d75533

```

4.2 Min Cut [d41d8c]

```

void policeChase(int n, Dinic<int> &d) {
    int ans = d.work(0, n - 1);
    vector<int> vis(n);
    [&](this auto &&self, int u) -> void {
        if (vis[u]) return;
        vis[u] = 1;
        for (int id : d.g[u]) {
            auto [to, f, cap] = d.e[id];
            if (cap - f > 0) self(to);
        }
    }(0);
    for (int i = 0; i < n; i++) {
        if (!vis[i]) continue;
        for (int id : d.g[i]) {
            if (id & 1) continue;
            auto e = d.e[id];
            if (!vis[e.to]) cout << i + 1 << " " << e.to + 1 << "\n";
        }
    }
}

```

4.3 MCMF [d41d8c]

```

template<class Tf, class Tc> struct MCMF {
    struct Edge {
        int to;
        Tf f, cap; // 流量跟容量
        Tc cost;
    };
    int n, m, s, t;
    const Tf INF_FLOW = numeric_limits<Tf>::max() / 2;
    const Tc INF_COST = numeric_limits<Tc>::max() / 2;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<Tc> dis;
    vector<int> rt, inq;
    MCMF(int n) : n(n), m(0), g(n) {}
    void addEdge(int u, int v, Tf cap, Tc cost) {
        e.push_back({v, 0, cap, cost});
        e.push_back({u, 0, 0, -cost});
        g[u].push_back(m++);
        g[v].push_back(m++);
    }
    bool spfa() { // O(VE)
        dis.assign(n, INF_COST);
        rt.assign(n, -1), inq.assign(n, 0);
        queue<int> q; q.push(s);
        dis[s] = 0, inq[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            for (int id : g[u]) {
                auto [v, f, cap, cost] = e[id];
                Tc ndis = dis[u] + cost;
                if (f < cap && dis[v] > ndis) {
                    dis[v] = ndis, rt[v] = id;
                    if (!inq[v])
                        q.push(v), inq[v] = 1;
                }
            }
        }
        return dis[t] != INF_COST;
    }
    // spfa: O(FVE), dijkstra: O(VE + FElogV)
    pair<Tf, Tc> work(int s_, int t_, Tf need) {
        s = s_, t = t_;
        Tf flow{}; Tc cost{};
        while (
            spfa() { // dijkstra: flow == 0 ? spfa() : dijkstra()
                Tf f = need;
                for (int i = t; i != s; i = e[rt[i] ^ 1].to)
                    f = min(f, e[rt[i]].cap - e[rt[i]].f);
                for (int i = t; i != s; i = e[rt[i] ^ 1].to)
                    e[rt[i]].f += f, e[rt[i] ^ 1].f -= f;
                flow += f, need -= f;
                cost += f * dis[t];
                if (need == 0) break;
            }
        )
        return {flow, cost};
    }
    /*
    vector<Tc> pot;
    bool dijkstra
        () { // O(ElogV), 用上一輪的 dis 當 pot, 邊權變非負
        pot = dis;
        dis.assign(n, INF_COST), rt.assign(n, -1);
        priority_queue
            <pair<Tc, int>, vector<pair<Tc, int>>, greater<>> pq;
        dis[s] = 0;
        pq.emplace(dis[s], s);
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();

```

```

    if (dis[u] < d) continue;
    for (int id : g[u]) {
        auto [v, f, cap, cost] = e[id];
        Tc ndis = dis[u] + cost + pot[u] - pot[v];
        if (f < cap && dis[v] > ndis) {
            dis[v] = ndis, rt[v] = id;
            pq.emplace(ndis, v);
        }
    }
}
for (int i = 0; i < n; i++)
    if (dis[i] != INF_COST) dis[i] += pot[i];
return dis[t] != INF_COST;
} */ // 54295b
}; // a224e5

```

4.4 Hungarian [d41d8c]

```

struct Hungarian { // 0-based, 0(VE)
    int n, m;
    vector<vector<int>> adj;
    vector<int> used, vis;
    vector<pair<int, int>> match;
    Hungarian(int n, int m) : n(n), m(m) {
        adj.assign(n + m, {});
        used.assign(n + m, -1);
        vis.assign(n + m, 0);
    }
    void addEdge(int u, int v) {
        adj[u].push_back(n + v);
        adj[n + v].push_back(u);
    }
    bool dfs(int u) {
        int sz = adj[u].size();
        for (int i = 0; i < sz; i++) {
            int v = adj[u][i];
            if (vis[v] == 0) {
                vis[v] = 1;
                if (used[v] == -1 || dfs(used[v])) {
                    used[v] = u;
                    return true;
                }
            }
        }
        return false;
    }
    vector<pair<int, int>> work() {
        match.clear();
        used.assign(n + m, -1);
        vis.assign(n + m, 0);
        for (int i = 0; i < n; i++) {
            fill(vis.begin(), vis.end(), 0);
            dfs(i);
        }
        for (int i = n; i < n + m; i++)
            if (used[i] != -1)
                match.emplace_back(used[i], i - n);
        return match;
    }
};

```

• 有向無環圖:

- 最小不相交路徑覆蓋: 最小路徑數 = 頂點數 - 最大匹配數
- 最小相交路徑覆蓋: 先用 Floyd 求傳遞封包, 有連邊就建邊, 然後再套最小不相交路徑覆蓋

• 二分圖:

- 最小點覆蓋: 選出一些點, 使每條邊至少有一個端點被選到的最少數量
- 最小點覆蓋 = 最大匹配數
- 還原解: flow 的作法是從源點開始 dfs, 只走 $\text{cap} - \text{flow} > 0$ 的邊, 最後挑選左邊還沒被跑過的點和右邊被跑過的點當作覆蓋的點

- 最大獨立集: 選出一些點, 使這些點兩兩沒有邊連接的最大數量
- 最大獨立集 = 點數 - 最大匹配數
- 還原解: 最小點覆蓋的補集

- 最少邊覆蓋: 選出一些邊, 讓所有點都覆蓋到的最少數量
- 最少邊覆蓋 = 點數 - 孤立點數 - 最大匹配數
- 還原解: 最大匹配 + 未被匹配的非孤立點隨便配

5 String

5.1 Hash [234076]

```

const int D = 59;
vector<int> rollingHash(string &s) {
    vector<int> a(0);
    for (auto c : s)
        a.push_back(mul(a.back(), D) + (c - 'A' + 1));
    return a;
}
int qryHash(vector<int> &h, int l, int r) { // [l, r)
    return sub(h[r], mul(h[l], power(D, r - l)));
}

```

5.2 KMP [cdf28b]

```

template<class T> struct KMP {
    int n;
    T pat;
    vector<int> f;
    // f 存匹配失敗時, 移去哪
    // 也就是 pat[0, i] 的最長共同前後綴長度
    // ex : a b c a b c
    //      -1 -1 -1 0 1 2
    KMP(const T &pat) : n(pat.size()), pat(pat), f(n, -1) {
        for (int i = 1; i < n; i++) {
            int j = f[i - 1];
            while (j != -1 && pat[j + 1] != pat[i]) j = f[j];
            if (pat[j + 1] == pat[i]) f[i] = j + 1;
        }
    }
    vector<int> work(const T &s) {
        vector<int> pos;
        for (int i = 0, j = -1; i < s.size(); i++) {
            while (j != -1 && pat[j + 1] != s[i]) j = f[j];
            if (pat[j + 1] == s[i]) j++;
            if (j + 1 == n) {
                pos.push_back(i - j);
                j = f[j];
            }
        }
        return pos;
    }
};

```

5.3 Z Function [a1f65d]

```

// z[i] 表示 s 和 s[i, n - 1] (以 s[i] 開頭的后綴)
// 的最長公共前綴 (LCP) 的長度
template<class T> vector<int> Z(const T &s) {
    int n = s.size();
    vector<int> z(n);
    z[0] = n; // lcp(s, s), -1 or n
    for (int i = 1, j = 1; i < n; i++) {
        z[i] = max(0, min(j + z[j] - i, z[i - j]));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > j + z[j]) j = i;
    }
    return z;
}

```

5.4 Manacher [9f76ee]

```

// 找到對於每個位置的迴文半徑
template<class T> vector<int> manacher(const T &s) {
    vector<int> t(-1);
    for (auto c : s) t.push_back(c), t.push_back(-1);
    int n = t.size();
    vector<int> r(n);
    for (int i = 0,
        j = 0; i < n; i++) { // i 是中心, j 是最長回文字串中心
        if (2 * j - i >= 0 && j + r[j] > i)
            r[i] = min(r[2 * j - i], j + r[j] - i);
        while (i - r[i]
            >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]])
            r[i]++;
        if (i + r[i] > j + r[j]) j = i;
    }
    return r;
}
// # a # b # a #
// 1 2 1 4 1 2 1
// # a # b # b # a #
// 1 2 1 2 5 2 1 2 1
// 值 -1 代表原回文字串長度
// (id - val + 1) / 2 可得原字串回文開頭

```

5.5 Trie [595441]

```

const int N = 1E7; // 0 -> initial state
const int ALPHABET_SIZE = 26;
int tot = 0;
int trie[N][ALPHABET_SIZE], cnt[N];
int newNode() {
    tot++;
    fill_n(trie[tot], ALPHABET_SIZE, 0);
    cnt[tot] = 0;
    return tot;
}
void reset() {
    tot = -1;
    newNode();
}
void add(const string &s, int i = 0, int p = 0) {
    if (i == s.size()) {
        cnt[p]++;
        return;
    }
    int &q = trie[p][s[i] - 'a'];
    if (!q) q = newNode();
    add(s, i + 1, q);
}
int find(const string &s, int i = 0, int p = 0) {
    if (i == s.size()) return cnt[p];
}

```

```
int q = trie[p][s[i] - 'a'];
if (!q) return 0;
return find(s, i + 1, q);
}
```

5.6 SA [9262af]

```
template<class T> struct SuffixArray {
    int n;
    vector<int> sa, rk, lc;
    // n: 字串長度
    // sa: 後綴數組, sa[i] 表示第 i 小的後綴的起始位置
    // rk: 排名數組, rk[i] 表示從位置 i 開始的後綴的排名
    // lc: LCP
    // 數組, lc[i] 表示 sa[i] 和 sa[i + 1] 的最長公共前綴長度
    SuffixArray(
        const T &s : n(s.size()), sa(n), lc(n - 1), rk(n) {
            iota(sa.begin(), sa.end(), 0);
            sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; });
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
            int k = 1;
            vector<int> tmp, cnt(n);
            tmp.reserve(n);
            while (rk[sa[n - 1]] < n - 1) {
                tmp.clear();
                for (int i = 0; i < k; i++) tmp.push_back(n - k + i);
                for (auto i : sa) if (i >= k) tmp.push_back(i - k);
                fill(cnt.begin(), cnt.end(), 0);
                for (int i = 0; i < n; i++) cnt[rk[i]]++;
                for (int i = 1; i < n; i++) cnt[i] += cnt[i - 1];
                for (int i =
                    n - 1; i >= 0; i--) sa[--cnt[rk[tmp[i]]]] = tmp[i];
                swap(rk, tmp); rk[sa[0]] = 0;
                for (int i = 1; i < n; i++) rk[sa[i]] = rk[
                    sa[i - 1]] + (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i -
                    1] + k == n || tmp[sa[i - 1]] + k < tmp[sa[i] + k]);
                k *= 2;
            }
            for (int i = 0, j = 0; i < n; i++) {
                if (rk[i] == 0) {
                    j = 0;
                } else {
                    for (j -= j > 0; i + j < n && sa[rk[i] - 1]
                        + j < n && s[i + j] == s[sa[rk[i] - 1] + j]; j++);
                    lc[rk[i] - 1] = j;
                }
            }
        };
    // ad4f20
    RMQ<int> rmq(sa, lc);
    auto lcp = [&](int i, int j) { // [i, j]
        i = sa.rk[i], j = sa.rk[j];
        if (i > j) swap(i, j);
        assert(i != j);
        return rmq(i, j);
    };
};
```

5.7 AC [d34486]

```
struct AC {
    static constexpr int ALPHABET_SIZE = 26;
    struct Node {
        int fail; // 指向最長後綴
        int cnt; // 有多少模式字串是自己的後綴
        array<int, ALPHABET_SIZE> ch, next;
        // next 是補全後的轉移
    };
    vector<Node> t;
    AC() : t(1) {}
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int insert(const string &s) {
        int u = 0;
        for (char c : s) {
            if (!t[u].ch[c - 'a']) {
                int v = newNode();
                t[u].ch[c - 'a'] = v;
            }
            u = t[u].ch[c - 'a'];
        }
        t[u].cnt++;
        return u;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < ALPHABET_SIZE; c++) {
            if (t[0].ch[c]) {
                q.push(t[0].ch[c]);
                t[0].next[c] = t[0].ch[c];
            }
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int c = 0; c < ALPHABET_SIZE; c++) {
                if (t[u].ch[c]) {
                    int v = t[u].ch[c];

```

```
                t[v].fail = t[t[u].fail].next[c];
                t[v].cnt += t[t[v].fail].cnt;
                t[u].next[c] = v;
                q.push(v);
            } else {
                t[u].next[c] = t[t[u].fail].next[c];
            }
        }
    }
};
```

5.8 SAM [83a519]

```
struct SAM {
    // 0 -> initial state
    static constexpr int ALPHABET_SIZE = 26;
    // node -> strings with the same endpos set
    // link -> longest suffix with different endpos set
    // len -> state's longest suffix
    // fpos -> first endpos
    // strlen range -> [len(link) + 1, len]
    struct Node {
        int len, link = -1, fpos;
        array<int, ALPHABET_SIZE> next;
    };
    vector<Node> t;
    SAM() : t(1) {}
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int extend(int p, int c) {
        int cur = newNode();
        t[cur].len = t[p].len + 1;
        t[cur].fpos = t[cur].len - 1;
        while (p != -1 && !t[p].next[c]) {
            t[p].next[c] = cur;
            p = t[p].link;
        }
        if (p == -1) {
            t[cur].link = 0;
        } else {
            int q = t[p].next[c];
            if (t[p].len + 1 == t[q].len) {
                t[cur].link = q;
            } else {
                int r = newNode();
                t[r].len = t[p].len + 1;
                t[r].link = t[p].link;
                while (p != -1 && t[p].next[c] == q) {
                    t[p].next[c] = r;
                    p = t[p].link;
                }
                t[q].link = t[cur].link = r;
            }
        }
        return cur;
    }
}; // 57198e
void solve(int n, string s, ll k) { // Substring Order II
    vector<int> last(n + 1);
    SAM sam;
    for (int i = 0; i < n; i++)
        last[i + 1] = sam.extend(last[i], s[i] - 'a');
    int sz = sam.t.size();

    vector<int> cnt(sz); // endpos size
    for (int i = 1; i <= n; i++) cnt[last[i]]++;
    vector<vector<int>> g(sz);
    for (int i = 1; i < sz; i++)
        g[sam.t[i].link].push_back(i);
    [&](this auto &&self, int u) -> void {
        for (auto v : g[u]) self(v), cnt[u] += cnt[v];
    } (0);

    vector<ll> dp(sz, -1);
    // for any path from
    // root, how many substring's prefix is the the path string
    [&](this auto &&self, int u) -> ll {
        if (dp[u] != -1) return dp[u];
        dp[u] = cnt[u]; // distinct: = 1
        for (int c = 0; c < SAM::ALPHABET_SIZE; c++) {
            int v = sam.t[u].next[c];
            if (v) dp[u] += self(v);
        }
        return dp[u];
    } (0);

    int p = 0; string ans;
    while (k > 0) { // 1-based
        for (int c = 0; c < SAM::ALPHABET_SIZE; c++) {
            int v = sam.t[p].next[c];
            if (v) {
                if (k > dp[v]) {
                    k -= dp[v];
                } else {
                    ans.push_back('a' + c);
                    k -= cnt[v]; // distinct: --
                    p = v; break;
                }
            }
        }
    }
}
```

```

    }
} cout << ans << "\n";
}

```

5.9 Palindrome Tree [1da7bb]

```

struct PAM {
    // 0 -> even root, 1 -> odd root
    static constexpr int ALPHABET_SIZE = 26;
    // fail -> longest prefix(suffix) palindrome
    // number end at i = end at link[last[i]] + 1
    struct Node {
        int len, fail, cnt;
        array<int, ALPHABET_SIZE> next;
        Node() : len{}, fail{}, next{} {}
    };
    vector<int> s;
    vector<Node> t;
    PAM() {
        t.assign(2, Node());
        t[0].len = 0, t[0].fail = 1;
        t[1].len = -1;
    }
    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }
    int getFail(int p, int i) {
        while (i - t[p].len < 1 || s[i - t[p].len - 1] != s[i])
            p = t[p].fail;
        return p;
    }
    int extend(int p, int c) {
        int i = s.size();
        s.push_back(c);
        p = getFail(p, i);
        if (!t[p].next[c]) {
            int r = newNode();
            int v = getFail(t[p].fail, i);
            t[r].len = t[p].len + 2;
            t[r].fail = t[v].next[c];
            t[p].next[c] = r;
        }
        return p = t[p].next[c];
    }
};

// string s; cin >> s;
// int n = s.length();
// vector<int> last(n + 1);
// last[0] = 1;
// PAM pam;
// for (int i = 0; i < n; i++)
//     last[i + 1] = pam.extend(last[i], s[i] - 'a');
// int sz = pam.t.size();
// vector<int> cnt(sz);
// for (int i = 1; i <= n; i++)
//     cnt[last[i]]++; // 去重 = 1
// for (int i = sz - 1; i > 1; i--)
//     cnt[pam.t[i].fail] += cnt[i];

```

5.10 Duval [f0a5c2]

```

// duval_algorithm
// 將字串分解成若干個非嚴格遞減的非嚴格遞增字串
template<class T> vector<T> duval(const T &s) {
    int i = 0, n = s.size();
    vector<T> res;
    while (i < n) {
        int k = i, j = i + 1;
        for (; j < n && s[k] <= s[j]; j++) {
            if (s[k] < s[j]) k = i;
            else k++;
        }
        while (i <= k) {
            res.emplace_back(s.begin() + i, s.begin() + i + j - k);
            i += j - k;
        }
    }
    return res;
}

// 最小旋轉字串
template<class T> T minRound(T s) {
    s.insert(s.end(), s.begin(), s.end());
    int i = 0, n = s.size(), start = i;
    while (i < n / 2) {
        start = i;
        int k = i, j = i + 1;
        for (; j < n && s[k] <= s[j]; j++) {
            if (s[k] < s[j]) k = i;
            else k++;
        }
        while (i <= k) i += j - k;
    }
    return {s.begin() + start, s.begin() + start + n / 2};
}

```

6 Math

6.1 Mint [78bb96]

```

// P * P > ll => mul in pollard rho
// P * 2 > int => x int to ll
const int P = 1E9 + 7;
template<class T> constexpr T power(T a, ll b) {
    T res{1};
    for (; b > 0; b >>= 1, a *= a)
        if (b & 1) res *= a;
    return res;
} // b2c151
struct Z { // simple
    int x;
    Z(ll v = 0) : x(v % P) { if (x < 0) x += P; }
    Z operator-() const { return P - x; }
    Z inv() const { return power(*this, P - 2); }

    Z &operator
    +=(Z a) { if ((x += a.x) >= P) x -= P; return *this; }
    Z &operator
    -=(Z a) { if ((x -= a.x) < 0) x += P; return *this; }
    Z &operator*=(Z a) { x = 1LL * x * a.x % P; return *this; }
    Z &operator/=(Z a) { return *this *= a.inv(); }

    friend Z operator+(Z a, Z b) { return a += b; }
    friend Z operator-(Z a, Z b) { return a -= b; }
    friend Z operator*(Z a, Z b) { return a *= b; }
    friend Z operator/(Z a, Z b) { return a /= b; }
    /* friend istream &operator>>(istream &is, Z &a) {
        ll v; is >> v;
        a = v; return is;
    } // 4a6a72
    friend ostream &operator<<(ostream &os, Z a) {
        return os << a.x;
    } // 81cfbc */
}; // f00eb5
template<int P> struct Mint {
    static inline int Mod = 998244353;
    static int p() { return P > 0 ? P : Mod; }
    static void setMod(int Mod_) { Mod = Mod_; }
    int x;
    Mint(ll v = 0) : x(v % p()) { if (x < 0) x += p(); }
    Mint operator-() const { return p() - x; }
    Mint inv() const { return power(*this, p() - 2); }

    Mint &operator+=(Mint
        a) { if ((x += a.x) >= p()) x -= p(); return *this; }
    Mint &operator-=(
        Mint a) { if ((x -= a.x) < 0) x += p(); return *this; }
    Mint &operator
    *=(Mint a) { x = 1LL * x * a.x % p(); return *this; }
    Mint &operator/=(Mint a) { return *this *= a.inv(); }

    friend Mint operator+(Mint a, Mint b) { return a += b; }
    friend Mint operator-(Mint a, Mint b) { return a -= b; }
    friend Mint operator*(Mint a, Mint b) { return a *= b; }
    friend Mint operator/(Mint a, Mint b) { return a /= b; }
    /* friend istream &operator>>(istream &is, Mint &a) {
        ll v; is >> v;
        a = v; return is;
    } // f964a7
    friend ostream &operator<<(ostream &os, Mint a) {
        return os << a.x;
    } // d01053 */
};
using Z = Mint<P>; // 426039

vector<Z> fac, invfac, inv;
void init(int n) {
    fac.resize(n + 1);
    invfac.resize(n + 1);
    inv.resize(n + 1);
    fac[0] = invfac[0] = 1;
    for (int i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i;
    }
    invfac[n] = fac[n].inv();
    for (int i = n; i > 0; i--) {
        invfac[i - 1] = invfac[i] * i;
        inv[i] = invfac[i] * fac[i - 1];
    }
} // c5dd4f
Z binom(int n, int m) {
    if (n < m || m < 0) return 0;
    return fac[n] * invfac[m] * invfac[n - m];
} // 9326d0
Z lucas(ll n, ll m) { // O(p + T log(n)), p is prime
    return m ? binom(n % P, m % P) * lucas(n / P, m / P) : 1;
} // de3d87

```

6.2 Sieve [ad6c6e]

```

vector<int> minp, primes, mu, phi, dnum, pnum;
vector<ll> dsum;
void sieve(int n) {
    minp.assign(n + 1, 0);
    phi.assign(n + 1, 0), mu.assign(n + 1, 0);
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
            mu[i] = -1;

```

```

    }
    for (int p : primes) {
        if (i * p > n) break;
        minp[i * p] = p;
        if (p == minp[i]) {
            phi[i * p] = phi[i] * p;
            mu[i * p] = 0;
            break;
        } else {
            phi[i * p] = phi[i] * (p - 1);
            mu[i * p] = -mu[i];
        }
    }
}
dnum.assign(n + 1, 0), dsum.assign(n + 1, 0);
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i) dnum[j]++, dsum[j] += i;
pnum.assign(n + 1, 0);
for (int
    p : primes) for (int j = p; j <= n; j += p) pnum[j]++;
}

```

6.3 Matrix [106d8e]

```

template<class T> struct Mat {
    int n = 0, m = 0; // n == 0: identity of any size
    vector<vector<T>>> a;
    Mat(int) {} // for power's T res[1]
    Mat(int n, int m) : n(n), m(m), a(n, vector<T>(m)) {}
    Mat(const vector
        <vector<T>>> &a) : n(a.size()), m(a[0].size()), a(a) {}
    Mat operator*(const Mat &b) const {
        if (!n) return b;
        if (!b.n) return *this;
        assert(m == b.n);
        Mat res(n, b.m);
        for (int i = 0; i < n; i++)
            for (int l = 0; l < m; l++)
                for (int j = 0; j < b.m; j++)
                    res.a[i][j] += a[i][l] * b.a[l][j];
        return res;
    }
    Mat &operator*=(const Mat &b) { return *this = *this * b; }
};

```

6.4 Miller Rabin Pollard Rho [394cfb]

```

ll mul(ll a, ll b, ll p) {
    ll res = a * b - ll(1.L * a * b / p) * p;
    res %= p;
    if (res < 0) res += p;
    return res;
}
ll power(ll a, ll b, ll p) {
    ll res[1];
    for (; b; b /= 2, a = mul(a, a, p))
        if (b & 1) res = mul(res, a, p);
    return res;
}
vector<
    ll> chk[2, 325, 9375, 28178, 450775, 9780504, 1795265022];
bool check(ll a, ll d, int s, ll n) {
    a = power(a, d, n);
    if (a <= 1) return 1;
    for (int i = 0; i < s; i++, a = mul(a, a, n)) {
        if (a == 1) return 0;
        if (a == n - 1) return 1;
    }
    return 0;
}
bool isPrime(ll n) {
    if (n < 2) return 0;
    if (n % 2 == 0) return n == 2;
    ll d = n - 1, s = 0;
    while (d % 2 == 0) d /= 2, s++;
    for (ll i : chk)
        if (!check(i, d, s, n)) return 0;
    return 1;
}
const vector<ll> small = {2, 3, 5, 7, 11, 13, 17, 19};
ll findFactor(ll n) {
    if (isPrime(n)) return 1;
    for (ll p : small)
        if (n % p == 0) return p;
    ll x, y = 2, d, t = 1;
    auto f = [&](ll a) {
        return (mul(a, a, n) + t) % n;
    };
    for (int l = 2; ; l *= 2) {
        x = y;
        int m = min(l, 32);
        for (int i = 0; i < l; i += m) {
            d = 1;
            for (int j = 0; j < m; j++)
                y = f(y), d = mul(d, abs(x - y), n);
            ll g = __gcd(d, n);
            if (g == n) {
                l = 1, y = 2, ++t;
                break;
            }
            if (g != 1) return g;
        }
    }
}

```

```

    }
    map<ll, int> res;
    void pollardRho(ll n) {
        if (n == 1) return;
        if (isPrime(n)) {
            res[n]++;
            return;
        }
        ll d = findFactor(n);
        pollardRho(n / d), pollardRho(d);
    }
}

```

6.5 Primitive Root [b0ba96]

```

int findPrimitiveRoot(ll m) {
    Mlong<0>::setMod(m); // Mlong if needed
    ll phi = m; // m - 1 if m prime
    res.clear();
    pollardRho(m);
    for (auto [p, _] : res) phi = phi / p * (p - 1);
    vector<ll> pr; // prime factors of phi
    res.clear();
    pollardRho(phi);
    for (auto &[p, _] : res) pr.push_back(p);
    for (int i = 1; i <= m; i++) {
        bool ok = true;
        for (int j = 0; j < int(pr.size()) && ok; j++)
            ok &= power(Mlong<0>(i), phi / pr[j]).x != 1;
        if (ok) return i;
    }
    return -1; // m != 1, 2, 4, p^k, 2(p^k)
}

```

6.6 CRT [35109b]

```

// ax = b (mod m) 的前提是 gcd(a, m) | b
// Ax + By = C (g = gcd(A, B))
// (A/g) * x + (B/g) * y = C/g
// solve (A/g) * x' + (B/g) * y' = 1
// x = (x' * (C/g)) + i(B/g)
pair<ll, ll> exgcd(ll a, ll b) {
    if (b == 0) return {1, 0};
    auto [y, x] = exgcd(b, a % b);
    return {x, y - (a / b) * x};
}
// 結果可能為負，注意正負化
// Smallest non-negative solution
using i128 = __int128_t;
pair<ll, ll> CRT(ll r1, ll m1, ll r2, ll m2) {
    ll g = __gcd(m1, m2);
    if ((r2 - r1) % g) return {-1, g};
    m1 /= g, m2 /= g;
    auto [p1, p2] = exgcd(m1, m2);
    i128 lcm = i128(m1) * m2 * g;
    i128 res = i128(p1) * (r2 - r1) * m1 + r1;
    return {(res % lcm + lcm) % lcm, lcm};
}
// 8ddc68
pair<ll, ll> EXCRT(vector<pair<int, int>> a) {
    ll R = 0, M = 1;
    for (auto [r, m] : a) {
        auto [res, lcm] = CRT(R, M, r, m);
        if (res == -1) return {-1, -1};
        R = res, M = lcm;
    }
    return {R, M};
}
// 3396d1
// gcd(mod) = 1, support 3 1E9 Mod
pair<i128, i128> CRT(vector<pair<int, int>> a) {
    i128 s = 1, res = 0;
    for (auto [r, m] : a) s *= m;
    for (auto [r, m] : a) {
        i128 t = s / m;
        res = (res + r * t % s * exgcd(t, m).first % s) % s;
    }
    return {(res + s) % s, s};
}
// 8ffc2d

```

6.7 XOR Basis [02f0c0]

```

auto add = [&](vector<int> &bas, int x) {
    for (auto i : bas) x = min(x, x ^ i);
    if (x) bas.push_back(x);
};
sort(bas.begin(), bas.end()); // 最簡化列梯
for (auto i = bas.begin(); i != bas.end(); i++) {
    for (auto j = next(i); j != bas.end(); j++) {
        *j = min(*j, *j ^ *i);
    }
}
// [l, r] 的區間 xor max (1-indexed)
vector<array<int, B>> bas(n + 1), lst(n + 1);
for (int i = 1; i <= n; i++) {
    int x = a[i - 1], p = i;
    bas[i] = bas[i - 1], lst[i] = lst[i - 1];
    for (int j = B - 1; j >= 0; j--)
        if (x >> j & 1) {
            if (!bas[i][j]) {
                bas[i][j] = x, lst[i][j] = p;
                break;
            }
            if (lst[i][j] < p)

```

```

        swap(lst[i][j], p), swap(x, bas[i][j]);
        x ^= bas[i][j];
    }
}
auto qry = [&](int l, int r) -> int {
    int mx = 0;
    for (int i = B - 1; i >= 0; i--)
        if (lst[r][i] >= l)
            mx = max(mx, mx ^ bas[r][i]);
    return mx;
};

```

6.8 Gaussian Elimination [5d1aa7]

```

// 找反矩陣
就開 2n，右邊放單位矩陣，做完檢查左半是不是單位，回傳右半
// 0 : no solution
// -1 : infinity solution
// 1 : one solution
template<class T> tuple<T,
    int, vector<T>> gaussianElimination(vector<vector<T>> a) {
    T det = 1;
    bool zeroDet = false;
    int n = a.size(), m = a[0].size(), rk = 0, sgn = 1;
    for (int c = 0; c < n; c++) {
        int p = -1;
        for (int r = rk; r < n; r++) {
            if (a[r][c] != 0) {
                p = r;
                break;
            }
        }
        if (p == -1) {
            zeroDet = true;
            continue;
        }
        if (p != rk) swap(a[rk], a[p]), sgn *= -1;
        det *= a[rk][c];
        T inv = 1 / a[rk][c];
        for (int j = c; j < m; j++) a[rk][j] *= inv;
        for (int r = 0; r < n; r++) {
            if (r == rk || a[r][c] == 0) continue;
            T fac = a[r][c];
            for (int j = c; j < m; j++)
                a[r][j] -= fac * a[rk][j];
        }
        rk++;
    }
    det = (zeroDet ? 0 : det * sgn);
    for (int r = rk; r < n; r++)
        if (a[r][m - 1] != 0) return {det, 0, {}};
    if (rk < n) return {det, -1, {}};
    vector<T> ans(n);
    for (int i = 0; i < n; i++) ans[i] = a[i][m - 1];
    return {det, 1, ans};
} // 0064b9
template<class T> tuple<int, vector<T>, vector<vector<T>>> findBasis(vector<vector<T>> a) {
    int n = a.size(), m = a[0].size(), rk = 0;
    vector<int> pos(m - 1, -1);
    for (int c = 0; c < m - 1; c++) {
        int p = -1;
        for (int r = rk; r < n; r++) {
            if (a[r][c] != 0) {
                p = r;
                break;
            }
        }
        if (p == -1) continue;
        if (p != rk) swap(a[rk], a[p]);
        pos[c] = rk;
        T inv = 1 / a[rk][c];
        for (int j = c; j < m; j++) a[rk][j] *= inv;
        for (int r = 0; r < n; r++) {
            if (r == rk || a[r][c] == 0) continue;
            T fac = a[r][c];
            for (int j = c; j < m; j++)
                a[r][j] -= fac * a[rk][j];
        }
        rk++;
    }
    vector<T> sol(m - 1);
    vector<vector<T>> basis;
    for (int r = rk; r < n; r++)
        if (a[r][m - 1] != 0)
            return {-1, sol, basis};
    for (int c = 0; c < m - 1; c++)
        if (pos[c] != -1)
            sol[c] = a[pos[c]][m - 1];
    for (int c = 0; c < m - 1; c++)
        if (pos[c] == -1) {
            vector<T> v(m - 1);
            v[c] = 1;
            for (int j = 0; j < m - 1; j++)
                if (pos[j] != -1)
                    v[j] = -a[pos[j]][c];
            basis.push_back(v);
        }
    return {rk, sol, basis};
} // 0e6dd7
template<class T> using Matrix = vector<vector<T>>;

```

6.9 Integer Partition [83bc9d]

```

// CSES_Sum_of_Divisors
const int Mod = 1E9 + 7;
const int inv_2 = 500000004;
// n / 1 + n / 2 + n / 3 + ... + n / n
void integerPartition() {
    ll ans = 0, n; cin >> n;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ll val = n / l; // n / l 到 n / r 一樣的值
        ll sum = ((l + r) % Mod) * ((r - l + 1) % Mod) % Mod * inv_2; // l 加到 r
        val %= Mod; sum %= Mod;
        ans += val * sum;
        ans %= Mod;
    }
    cout << ans << "\n";
}

```

6.10 Basic Theorems

- $a^{m-1} \equiv 1 \pmod{m}$
- $a^{m-2} \equiv a^{-1} \pmod{m}$
- $\tau(N) = \prod_{i=1}^k (p_i^0 + p_i^1 + \dots + p_i^{a_i})$
- $\prod_{d|N} d = N^{\tau(N)}$
- Highly Composite Number:
 - $(10^6): 720720 = 2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13, \tau = 240$
 - $(10^9): 735134400 = 2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17, \tau = 1344$
 - $(10^{12}): 963761198400 = 2^6 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23, \tau = 6720$
 - $(10^{15}): 866421317361600 = 2^6 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31, \tau = 26880$
 - $(10^{18}): 897612484786617600 = 2^8 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37, \tau = 103680$

6.10.1 Combination

- $C(n, m) = C(n, m-1) * (n-m+1) / m$
- $C(n+1, m) = C(n, m) + C(n, m-1)$
- $C(n, k) \equiv 1 \pmod{2} \Leftrightarrow$ all bit of $k \leq$ all bit of n in binary
- $(x+y)^n = \sum_{k=0}^n C(n, k) x^{n-k} y^k$

6.10.2 Game Theorem

- sg 值為 0 代表先手必敗
- 當前 sg 值 = 經過一方操作後，另一方所有可能的後繼狀態的 mex
- 若某個後繼狀態由多個狀態構成，就 xor 起來當作此後繼狀態的 sg 值
- 單組基礎 nim 的 sg 值為本身的原因: $f(0) = 0, f(1) = \text{mex}(f(0)) = 1, f(2) = \text{mex}(f(0), f(1)) = 2, \dots$ 都是自己
- 多組賽局可以把 sg 值 xor 起來，當成最後的 sg 值，nim 也是一樣，且由於 xor 性質，如果可以快速知道 $sg(1)g(2)\dots g(n)$ ，就可以用 xor 性質處理不連續組合

6.10.3 Pisano Period

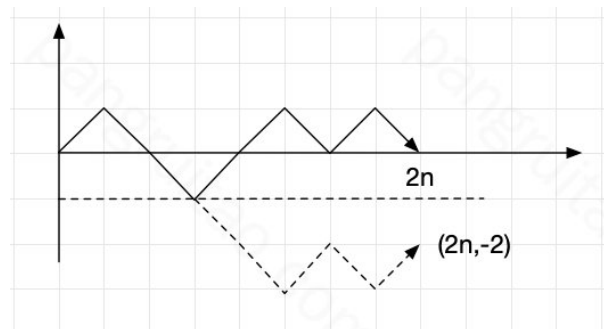
- $\pi(ab) = \text{lcm}(\pi(a), \pi(b)) \quad (\text{gcd}(a, b) = 1)$
- $\pi(p^e) | \pi(p) \cdot p^{e-1}$
- $\pi(p) | p^2 - 1 \quad (p \neq 2, 5)$
- $\pi(2) = 3, \pi(5) = 20$
- so can deal with $p \approx 10^9$ in long long

6.10.4 Burnside's lemma

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

- G : 各種翻轉操作所構成的置換群
- X/G : 本質不同的方案的集合
- X^g : 對於某一種操作 g ，所有方案中，經過 g 這種翻轉後保持不變的方案
- 集合取絕對值代表集合數

6.10.5 Catalan Theorem



1. n 個往上 n 個往下，先枚舉所有情況 $\frac{(2n)!}{n!n!} = C_n^{2n}$
 2. 扣掉非法的，有多少種可能讓最後的點落在 $(2n, -2)$
- 假設往上有 x 個，往下有 y 個，會有：

$$\begin{cases} x+y=2n \\ y-x=2 \end{cases} \Rightarrow \begin{cases} x=n-1 \\ y=n+1 \end{cases}$$

所以只要扣掉 C_{n-1}^{2n} 即可

6.10.6 Inversion

- 數論分塊可以快速計算一些含有除法向下取整的和式，就是像 $\sum_{i=1}^n f(i)g(\lfloor \frac{n}{i} \rfloor)$ 的和式。當可以在 $O(1)$ 內計算 $f(r) - f(l)$ 或已經預處理出 f 的前綴和時，數論分塊就可以在 $O(\sqrt{n})$ 的時間內計算上述和式的值。
- 迪利克雷捲積 $h(x) = \sum_{d|x} f(d)g(\frac{x}{d})$
- 積性函數
 - $\epsilon(n)$: 只在 $n=1$ 時為 1，其餘情況皆為 0。
 - 莫比烏斯函數 1.

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{for } n=1 \\ 0 & \text{for } n \neq 1 \end{cases}$$

- μ 是常數函數 1 的反元素

$$\Rightarrow \mu * 1 = \epsilon$$

- ϕ 歐拉函數: x 以下與 x 互質的數量

$$* \phi(p_1^{a_1} \cdots p_k^{a_k}) = (p_1^{a_1-1}(p_1-1)) \cdots (p_k^{a_k-1}(p_k-1))$$

$$* n = \sum_{d|n} \phi(d)$$

*

$$\phi * 1 = \sum_{d|n} \phi(\frac{n}{d}) \quad \text{質因數分解}$$

$$= \sum_{i=0}^c \phi(p^i)$$

$$= 1 + p^0(p-1) + p^1(p-1) + \dots + p^{c-1}(p-1)$$

$$= p^c$$

$$= id$$

- 莫比烏斯反演公式

$$- f(n) = \sum_{d|n} g(d) \Leftrightarrow g(n) = \sum_{d|n} \mu(d) f(\frac{n}{d})$$

$$- f(n) = \sum_{n|d} g(d) \Leftrightarrow g(n) = \sum_{n|d} \mu(\frac{d}{n}) f(d)$$

- 例子

$$\sum_{i=a}^b \sum_{j=c}^d [gcd(i,j) = k]$$

$$\Rightarrow \sum_{i=1}^x \sum_{j=1}^y [gcd(i,j) = k]$$

$$= \sum_{i=1}^{\lfloor \frac{x}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{y}{k} \rfloor} \epsilon(gcd(i,j))$$

$$= \sum_{i=1}^{\lfloor \frac{x}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{y}{k} \rfloor} \sum_{d|gcd(i,j)} \mu(d)$$

$$= \sum_{d=1}^{\infty} \mu(d) \sum_{i=1}^{\lfloor \frac{x}{kd} \rfloor} \sum_{j=1}^{\lfloor \frac{y}{kd} \rfloor} [d \text{ 可整除 } i \text{ 時為 } 1]$$

$$= \sum_{d=1}^{\min(\lfloor \frac{x}{k} \rfloor, \lfloor \frac{y}{k} \rfloor)} \mu(d) \lfloor \frac{x}{kd} \rfloor \lfloor \frac{y}{kd} \rfloor$$

6.11 Mobius Inverse [d41d8c]

```
void solve() { // pref: pref of mu
    ll a, b, c, d, k; cin >> a >> b >> c >> d >> k;
    sieve(N);
    auto cal = [&](ll x, ll y) -> int {
        int res = 0;
        for (int l = 1, r; l <= min(x, y); l = r + 1) {
            r = min(x / (x / l), y / (y / l));
            res += (pref[r] - pref[l - 1]) * (x / l) * (y / l);
        }
        return res;
    };
    cout <<
        cal(b / k, d / k) - cal((a - 1) / k, d / k) - cal(b /
        k, (c - 1) / k) + cal((a - 1) / k, (c - 1) / k) << "\n";
}
```

6.12 LCM Convolution [a64dbb]

```
// ak = sum_{d|k} a[d] (zeta, a \ast 1)
// ck = sum_{d|k} mu[d] * a[k/d] (inv zeta, a \ast mu)
template
<class T> vector<T> zeta(vector<T> a, bool inv = false) {
    int n = a.size() - 1;
    for (int p : primes)
        if (inv) for (int i = n / p; i >= 1; i--) a[i * p] -= a[i];
        else for (int i = 1; i <= n / p; i++) a[i * p] += a[i];
    return a;
}
// less implementation
if (inv) for (int i = 1; i <= n; i++)
    for (int j = 2 * i; j <= n; j += i) a[j] -= a[i];
```

```
else for (int i = n; i >= 1; i--)
    for (int j = 2 * i; j <= n; j += i) a[j] += a[i];
}
// ck = sum_{lcm(i,j)=k} a[i] * b[j]
template<class T> vector<T> conv(vector<T> a, vector<T> b) {
    a = zeta(a), b = zeta(b);
    for (int i = 1; i < a.size(); i++) a[i] *= b[i];
    return zeta(a, true);
}
```

6.13 exLucas [f8047f]

```
ll legendre(ll n, int p) { // n! 中質數 p 的幕次
    ll res = 0;
    while (n) { n /= p, res += n; }
    return res;
}
Z C_pe(ll n, ll m, int p, int e) {
    int pe = power(p, e);
    Z::setMod(pe); // inv 要用 exgcd
    vector<Z> fac(pe); fac[0] = 1;
    for (int j
        = 1; j < pe; j++) fac[j] = fac[j - 1] * (j % p ? j : 1);
    Z wilson =
        p == 2 && e >= 3 ? 1 : -1; // (p^e)! = wilson (mod p^e)
    function<Z(ll)> fastFac = [&](ll n)
    { return n ? power
        (wilson, n / pe) * fac[n % pe] * fastFac(n / p) : 1; };
    ll vp = legendre(n, p) - legendre(m, p) - legendre(n - m, p);
    if (vp >= e) return 0;
    return power
        (Z(p), vp) * fastFac(n) / (fastFac(m) * fastFac(n - m));
}
ll exlucas(ll n, ll m, int mod) {
    vector<pair<int, int>> a;
    for (int i = 2; i * i <= mod; i++) {
        if (mod % i) continue;
        int e = 0, p = i;
        while (mod % i == 0) mod /= i, e++, p *= i;
        a.emplace_back(C_pe(n, m, i, e).X, p);
    }
    if (mod != 1) a.emplace_back(C_pe(n, m, mod, 1).X, mod);
    return CRT(a);
}
```

6.14 Quadratic Residue [db1f2f]

```
int jacobi(int x, int p) {
    int s = 1;
    for (; p > 1; ) {
        x %= p;
        if (x == 0) return 0;
        const int r = __builtin_ctz(x);
        if ((r & 1) && ((p + 2) & 4)) s = -s;
        x >>= r;
        if (x & p & 2) s = -s;
        swap(x, p);
    }
    return s;
}
template<class Z> int quadraticResidue(Z x) {
    int p = Z::p();
    if (p == 2) return x.x & 1;
    const int jc = jacobi(x.x, p);
    if (jc == 0) return 0;
    if (jc == -1) return -1;
    Z b, d;
    while (true) {
        b = rand(), d = b * b - x;
        if (jacobi(d.x, p) == -1) break;
    }
    Z f0 = b, f1 = 1, g0 = 1, g1 = 0, tmp;
    for (int e = (p + 1) >> 1; e; e >>= 1) {
        if (e & 1) {
            tmp = g0 * f0 + d * (g1 * f1);
            g1 = g0 * f1 + g1 * f0, g0 = tmp;
        }
        tmp = f0 * f0 + d * (f1 * f1);
        f1 = f0 * f1 * 2, f0 = tmp;
    }
    return min(g0.x, p - g0.x);
}
```

6.15 BSGS [615b88]

```
// a^x = b (mod m)
// x = A * sq - B (0 <= A, B <= sq)
vector<ll> BSGS(int a, int b, int m) { // gcd(a, m) = 1
    Z::setMod(m);
    vector<ll> res; // O(sq)
    unordered_map<int, int> mp;
    ll sq = 1; while (sq * sq < m) sq++;
    Z rhs = b;
    for (int B = 0; B <= sq; B++) {
        mp[rhs.x] = B;
        rhs *= a;
    }
    Z a_pow_sq = power(Z(a), sq), lhs = a_pow_sq;
    for (int A = 1; A <= sq; A++) lhs *= a_pow_sq
        if (mp.find(lhs.x) != mp.end()) {
            int B = mp[lhs.x];
            res.push_back(A * sq - B);
        }
```

```

    }
    return res;
} // 468f7b
int exBSGS(int a, int b, int m) {
    if (b == 1 || m == 1) return 0;
    if (!a) return b ? -1 : 1;
    int k = 0;
    ll tmp = 1;
    while (__gcd(a, m) != 1) {
        int d = __gcd(a, m);
        if (b % d != 0) return -1;
        b /= d, m /= d, k++;
        tmp = tmp * (a / d) % m;
        if (tmp == b) return k;
    }
    Z::setMod(m);
    Z b_ = Z(b) * Z(tmp).inv();
    auto xs = BSGS(a, b_ * x, m);
    return xs
        .empty() ? -1 : *min_element(xs.begin(), xs.end()) + k;
} // 3ed312

```

7 Search and Greedy

7.1 Binary Search [d41d8c]

```

void binarySearch() {
    // 二分找上界
    // 如果無解會 = 原 lo, lo 要先 - 1
    while (lo < hi) {
        int x = (lo + hi + 1) / 2;
        if (check(x)) lo = x;
        else hi = x - 1;
    }
    cout << lo;
    // 二分找下界
    // 如果無解會 = 原 hi, hi 要先 + 1
    while (lo < hi) {
        int x = (lo + hi) / 2;
        if (check(m)) hi = x;
        else lo = x + 1;
    }
    cout << lo;
}

```

7.2 Ternary Search [d41d8c]

```

void ternarySearch() {
    int lo = 0, hi = 10;
    while (lo < hi) {
        int xl = lo + (hi - lo) / 3;
        int xr = hi - (hi - lo) / 3;
        int resl = calc(xl), resr = calc(xr);
        if (resl < resr) {
            lo = xl + 1;
        } else {
            hi = xr - 1;
        }
    }
}

```

8 DP

8.1 Projects [2ec0af]

```

// 排程有權重問題(strict)，價值最多且時間最少
void projects(int n, vector<array<int, 4>> a) {
    // a[i] = {t, s, w, idx}
    sort(a.begin(), a.end());
    vector<ll> ts(n + 1); // ensure ts[0] is minimum
    for (int i = 1; i <= n; i++) ts[i] = a[i][0];
    vector<array<ll, 3>> dp(n + 1); // {-w, time, 上個是誰}
    for (int i = 1; i <= n; i++) {
        int id = --lower_bound(all(ts), a[i][1]) - ts.begin();
        dp[i] = {dp[id - 1][0], dp[id - 1][1], -1};
        ll nw = dp[id][0] - a[i][2];
        ll nt = dp[id][1] + a[i][0] - a[i][1];
        chmin(dp[i], {nw, nt, id});
    }
    vector<int> used;
    for (int i = n; i != 0; i--) {
        if (dp[i][2] != -1) {
            used.push_back(a[i][3]);
            i = dp[i][2];
        } else i--;
    }
    reverse(used.begin(), used.end());
}

```

8.2 Bitmask [eedcfc]

```

void hamiltonianPath() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[--u].push_back(--v);
    }
    // 以...為終點，走過...
    vector dp(n, vector<int>(1 << n));
    dp[0][1] = 1;

```

```

    for (int mask = 1; mask < 1 << n; mask++) {
        if ((mask & 1) == 0) continue;
        for (int i = 0; i < n; i++) {
            if ((mask >> i & 1) == 0) continue;
            if (i == n - 1 && mask != (1 << n) - 1) continue;
            int pre = mask ^ (1 << i);
            for (int j : adj[i]) {
                if ((pre >> j & 1) == 0) continue;
                dp[i][mask] = (dp[i][mask] + dp[j][pre]) % Mod;
            }
        }
    }
    cout << dp[n - 1][(1 << n) - 1] << "\n";
}

void minClique() { // 移掉一些邊，讓整張圖由最少團組成
    int n, m;
    cin >> n >> m;
    vector<bitset<N>> g(n);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        u--; v--; g[u][v] = g[v][u] = 1;
    }
    vector<int> dp(1 << n, inf);
    dp[0] = 1;
    for (int mask = 0; mask < 1 << n; mask++) { // 先正常 dp
        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                int pre = mask ^ (1 << i);
                if (dp[pre] == 1 && (g[i] & bitset<N>(pre)) == pre)
                    dp[mask] = 1; // i 有連到所有 pre
            }
        }
    }
    for (int mask = 0; mask < 1 << n; mask++) // 然後枚舉子集 dp
        for (int sub = mask; sub; --sub &= mask)
            dp[mask] = min(dp[mask], dp[sub] + dp[mask ^ sub]);
    cout << dp[(1 << n) - 1] << "\n";
}

```

8.3 Monotonic Queue [c9ba14]

```

// 應用: dp(i) = h(i) + max(A(j)), for l(i) <= j <= r(i)
// A(j) 可能包含 dp(j), h(i) 可 O(1)
void boundedKnapsack() {
    int n, k; // 0(nk)
    vector<int> w(n), v(n), num(n);
    deque<int> q;
    // 將同餘的數分在同一組
    // 每次取出連續 num[i] 格中最大值
    // g_x = max(_{k=0}^num[i] (g'_{x-k} + v_i * k))
    // G_x = g'_{x} - v_i * x
    // x 代 x-k => v_i * (x-k)
    // g_x = max(_{k=0}^num[i] (G_{x-k} + v_i * x))
    vector<vector<ll>> dp(2, vector<ll>(k + 1));
    for (int i = 0; i < n; i++) {
        for (int r = 0; r < w[i]; r++) { // 餘數
            q.clear(); // q 記錄在 x = i 時的 dp 有單調性
            for (int x = 0; x * w[i] + r <= k; x++) {
                while (!q.empty() &&
                    q.front() < x - num[i]) q.pop_front(); // 維護遞減
                ll nxt = dp[0][x * w[i] + r] - x * v[i];
                while (!q.empty() && dp[0][q.back()]
                    * w[i] + r] - q.back() * v[i] < nxt) q.pop_back();
                q.push_back(x);
                dp[1][x * w[i] + r] = dp[0][q.
                    front() * w[i] + r] - q.front() * v[i] + x * v[i];
            }
            swap(dp[0], dp[1]);
        }
        cout << dp[0][k] << "\n";
    }
}

```

8.4 Digit [1c2af1]

```

// 只含 0, 1, 8 的回文數
const int N = 44;
ll memo[N + 1][N + 1]; // memset -1, once
ll solve(string s) {
    vector<int> a{0};
    while (!s.empty()) {
        a.push_back(s.back() - '0');
        s.pop_back();
    }
    int len = a.size() - 1;
    vector<int> now(len + 1, -1); // 紀錄目前的數字
    return [&](this
        auto &&self, int p, int eff, bool f0, bool lim) -> ll {
        if (!p) return 1; // eff: 扣除前導 0 的長度
        if (!lim && !f0 && memo[p][eff] != -1) return memo[p][eff];
        ll res = 0;
        int lst = lim ? a[p] : 9; // or 1 for binary
        for (int i = 0; i <= lst; i++) {
            if (i != 0 && i != 1 && i != 8) continue;
            bool nlim = lim && i == lst;
            now[p] = i;
            if (f0 && i == 0) { // 處理前導零
                res += self(p - 1, eff - 1, true, nlim);
            } else if (p > eff / 2) { // 前半段

```

```

    res += self(p - 1, eff, false, nlim);
} else if (now[p] == now[eff - p + 1]) {
    res += self(p - 1, eff, false, nlim);
}
}
if (!lim && !f0) memo[p][eff] = res;
return res;
} (len, len, true, true);
}
ll memo[N + 1][10]; // memset -1, once
// 不貼上限、
    不前導零、枚舉 p 往後所有數字中，每個 digit x 出現的次數
ll solve(int x, ll n) {
    vector<int> a{0};
    vector<ll> sum{0}; // 貼上限時，位數 p 以下有多少數字
    while (n) {
        a.push_back(n % 10);
        sum.push_back(
            (sum.back() + (n % 10) * power(10, a.size() - 2));
        n /= 10;
    }
    int len = a.size() - 1;
    return
        [&](this auto &&self, int p, bool f0, bool lim) -> ll {
            if (!p) return 0; // 貢獻來自於當前 digit
            if (!lim && !f0 && memo[p][x] != -1) return memo[p][x];
            ll res = 0;
            int lst = lim ? a[p] : 9; // or 1 for binary
            for (int i = 0; i <= lst; i++) {
                bool nlim = lim && i == lst;
                if (f0 && i == 0) { // 處理前導零
                    res += self(p - 1, true, nlim);
                } else if (nlim) { // 枚舉的前幾位都貼在 n 上
                    res += self(p - 1, false, nlim);
                    if (i == x) res += sum[p - 1] + 1;
                } else {
                    res += self(p - 1, false, nlim);
                    if (i == x) res += power(10, p - 1);
                }
            }
            if (!lim && !f0) memo[p][x] = res;
            return res;
        } (len, true, true);
}
// 連續兩位 digit > 1
ll memo[N + 1][10]; // memset -1, once
// 不貼上限、
    不前導零、前一位為 prv 時、枚舉 p 往後所有數字的合法個數
ll solve(ll x) { // 建 a...
    return [&](this
        auto &&self, int p, int prv, bool f0, bool lim) -> ll {
            if (!p) return 1; // 貢獻來自於當前數字數量
            if (!lim && !f0 && memo[p][prv] != -1) return memo[p][prv];
            ll res = 0;
            int lst = lim ? a[p] : 9; // or 1 for binary
            for (int i = 0; i <= lst; i++) {
                if (prv != -1 && abs(i - prv) < 2) continue;
                bool nlim = lim && i == lst;
                if (f0 && i == 0) { // 處理前導零
                    res += self(p - 1, -1, true, nlim);
                } else {
                    res += self(p - 1, i, false, nlim);
                }
            }
            if (!lim && !f0) memo[p][prv] = res;
            return res;
        } (len, -1, true, true);
}

```

8.5 SOS [ec9d5a]

// 使用情況：跟 bit 與(被)包含有關，且 x 在 1E6 左右
 // 題目：一數組，問有多少所有數 & 起來為 0 的集合數
 // dp[x] 代表包含 x 的 y 個數(比 x 大且 bit 1 全包含 x 的有幾個)
 // 答案應該包含在 dp[0] 內，但是有重複元素，所以考慮容斥
 // => ans = $\sum_{i=0}^n (-1)^{\text{pop_count}(i)} 2^{\text{dp}[i]-1}$
 // => 全
 部為 0 的個數 - 至少一個為 1 的個數 + 至少兩個為 1 的個數

```

void solve(int n, vector<int> a) {
    int m = __lg(*max_element(a.begin(), a.end())) + 1;
    // 定義 dp[mask] 為 mask 被包含於 a[i] 的 i 個數
    vector<Z> dp(1 << m);
    for (int i = 0; i < n; i++) dp[a[i]] += 1;
    for (int i = 0; i < m; i++) {
        for (int mask = 0; mask < 1 << m; mask++) {
            if (mask >> i & 1) {
                int pre = mask ^ (1 << i);
                dp[pre] += dp[mask];
            }
        }
    }
    for (int i = 0; i < 1 << m; i++) {
        dp[i] = power(Z(2), int(dp[i])) - 1;
    }
    // 排容，dp[i]：& 起來為 i 的集合數
    for (int i = 0; i < m; i++) {
        for (int mask = 0; mask < 1 << m; mask++) {

```

```

            if (mask >> i & 1) {
                int pre = mask ^ (1 << i);
                dp[pre] -= dp[mask];
            }
        }
    }
}
// x | y = x, 代表包含於 x 的 y 個數，定義為 dp[x][0]
// x & y = x, 代表包含 x 的 y 個數，定義為 dp[x][1]
// x & y
    != 0, 代表至少有一個位元都為 1 的 y 個數，= n - dp[~x][0]
void solve(int n, vector<int> a) {
    int m = __lg(*max_element(a.begin(), a.end())) + 1;
    vector<array<ll, 2>> dp(1 << m);
    for (int i = 0; i < n; i++) dp[a[i]][0]++, dp[a[i]][1]++;
    for (int i = 0; i < m; i++) {
        for (int mask = 0; mask < 1 << m; mask++) {
            if (mask >> i & 1) {
                int pre = mask ^ (1 << i);
                dp[mask][0] += dp[pre][0];
                dp[pre][1] += dp[mask][1];
            }
        }
    }
}

```

8.6 CHT [bdb950]

// 應用：dp(x) = C(x) + min/max(A(i) * x + B(i)), for i < x
 // 斜率遞減，求 min，上凸包，自行轉化問題
 // ex: m 遞增取 max => max(mx + b) = -min(-m x - b)

```

struct Line { // x 盡量從 1 開始
    ll m, b;
    Line(ll m = 0, ll b = 0) : m(m), b(b) {}
    ll eval(ll x) { return m * x + b; }
};

struct CHT { // 斜率單調
    deque<Line> hull;
    CHT() {}
    CHT(Line init) { addLine(init); }
    bool frontBad(Line &l1, Line &l2, ll x) {
        return l1.eval(x) >= l2.eval(x);
    }
    bool backBad(Line &l1, Line &l2, Line &l) {
        // 只要
        l2 跟 l 的 x 交點 <= l1 跟 l 的 x 交點，l2 就用不到了
        return (l1.b
            - l2.b) * (l1.m - l.m) <= (l.b - l1.b) * (l2.m - l.m);
    }
    void addLine(Line l) {
        if (hull.size() > 0 && hull.back().m == l.m) {
            if (hull.back().b <= l.b) return;
            else hull.pop_back();
        }
        while (hull.size() > 1 && backBad(hull
            [hull.size() - 2], hull.back(), l)) hull.pop_back();
        hull.push_back(l);
    }
    ll query(ll x) { // 查詢沒單調性需要二分搜
        while (hull.size() >
            0 && frontBad(hull[0], hull[1], x)) hull.pop_front();
        return hull[0].eval(x);
    }
};

```

8.7 DNC [a29240]

// 應用：切 k 段問題，且滿足四邊形不等式
 // w(a,c) + w(b,d) <= w(a,d) + w(b,c)
 // dp[k][j] = min(dp[k - 1][i] + cost[i][j])
 // O(K N logN)

```

auto getCost = [&](int l, int r) {}; // (l, r], 1-based
vector<ll> dp(n + 1, inf); dp[0] = 0;
for (int j = 0; j < k; j++) {
    vector<ll> ndp(n + 1, inf);
    [&](this auto &&self, int l, int r, int optl, int optr) {
        if (l > r) return;
        int m = (l + r) / 2, opt = 0;
        ndp[m] = inf;
        for (int i = optl; i <= min(m - 1, optr); i++) {
            ll x = dp[i] + getCost(i, m);
            if (x < ndp[m]) ndp[m] = x, opt = i;
        }
        self(l, m - 1, optl, opt);
        self(m + 1, r, opt, optr);
    } (1, n, j, n - 1);
    dp = ndp;
}

```

8.8 LiChao Segment Tree [2a9325]

// 應用：dp(i) = h(i) + min/max(A(j)X(i) + B(j)), for j <= r(i)
 // y = c + m x + b

```

template<class T, class F = less<ll>>
struct LiChaoSeg {
    F cmp = F();
    static const T inf = max(numeric_limits
        <T>::lowest() / 2, numeric_limits<T>::max() / 2, F());
    struct Line {
        T m, b;

```

```

    Line(T m = 0, T b = inf) : m(m), b(b) {}
    T eval(T x) const { return m * x + b; }
};
struct Node {
    Line line;
    ll l = -1, r = -1;
};
ll n;
vector<Node> nd;
LiChaoSeg(ll n) : n(n) { newNode(); }
void addLine(Line line) { update(0, 0, n, line); }
void rangeAddLine(Line line, ll ql, ll qr) { rangeUpdate(0, 0, n, ql, qr, line); }
T query(ll x) { return query(x, 0, 0, n); }
private:
    int newNode() {
        nd.emplace_back();
        return nd.size() - 1;
    }
    void update(int p, ll l, ll r, Line line) {
        ll m = (l + r) / 2;
        bool left = cmp(line.eval(l), nd[p].line.eval(l));
        bool mid = cmp(line.eval(m), nd[p].line.eval(m));
        if (mid) swap(nd[p].line, line);
        if (r - l == 1) return;
        if (left != mid) {
            if (nd[p].l == -1) nd[p].l = newNode();
            update(nd[p].l, l, m, line);
        } else {
            if (nd[p].r == -1) nd[p].r = newNode();
            update(nd[p].r, m, r, line);
        }
    }
    void rangeUpdate(int p, ll l, ll r, ll ql, ll qr, Line line) {
        if (r <= ql || l >= qr) return;
        if (ql <= l && r <= qr) return update(p, l, r, line);
        if (nd[p].l == -1) nd[p].l = newNode();
        if (nd[p].r == -1) nd[p].r = newNode();
        ll m = (l + r) / 2;
        rangeUpdate(nd[p].l, l, m, ql, qr, line);
        rangeUpdate(nd[p].r, m, r, ql, qr, line);
    }
    T query(ll x, int p, ll l, ll r) {
        if (p == -1) return inf;
        ll m = (l + r) / 2;
        if (x < m) return min(nd[p].line.eval(x), query(x, nd[p].l, l, m), cmp);
        else return min(nd[p].line.eval(x), query(x, nd[p].r, m, r), cmp);
    }
};

```

9 Geometry

9.1 Basic [eaf283]

```

const double eps = 1E-9;
int sign(double x) { return fabs(x) <= eps ? 0 : (x > 0 ? 1 : -1); }
template<class T> struct Pt {
    T x, y;
    Pt(T x = 0, T y = 0) : x(x), y(y) {}
    Pt operator-(const Pt& p) const { return Pt(-x, -y); }
    Pt operator+(const Pt& p) const { return Pt(x + p.x, y + p.y); }
    Pt operator-(const Pt& p) const { return Pt(x - p.x, y - p.y); }
    Pt operator*(const T k) const { return Pt(x * k, y * k); }
    Pt operator/(const T k) const { return Pt(x / k, y / k); }
    bool operator<(const Pt& p) const { return sign(x - p.x) == 0 ? y < p.y : x < p.x; }
    bool operator==(const Pt& p) const { return x == p.x && y == p.y; }
    bool operator!=(const Pt& p) const { return x != p.x || y != p.y; }
};
using P = Pt<double>; // 4cc20f
istream & operator>>(istream &is, P &p) { return is >> p.x >> p.y; }
ostream & operator<<(ostream &os, const P &p) {
    return os << "(" << p.x << ", " << p.y << ")";
} // 716e78

struct Line { P a, b; };
template<class T>
    T dot(const Pt<T> &a, const Pt<T> &b) { return a.x * b.x + a.y * b.y; }
template<class T> T
    cross(const Pt<T> &a, const Pt<T> &b) { return a.x * b.y - a.y * b.x; }
template<class T> T abs2(const Pt<T> &p) { return dot(p, p); }
double abs(P p) { return sqrt(abs2(p)); }
double abs(Line l) { return abs(l.a - l.b); }
int dir(P p, Line l) // left -1, right 1, on 0
{ return -sign(cross(l.b - l.a, p - l.a)); }
bool btw(P p, Line l) // c on segment ab?
{ return dir(p, l) == 0 && sign(dot(p - l.a, p - l.b)) <= 0; }
P norm(P p) { return p / abs(p); }
P rot(P p) { return {-p.y, p.x}; } // 90 degree CCW
P rot(P p, double d) { // CCW, d = degree * PI / 180
    double c = cos(d), s = sin(d);
    return P(p.x * c - p.y * s, p.x * s + p.y * c);
} // 48cbe0

bool parallel(Line l1, Line l2)
{ return sign(cross(l1.b - l1.a, l2.b - l2.a)) == 0; }
P lineIntersection(Line l1, Line l2)

```

```

{ return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b)); }
P projvec(P p, Line l) {
    P v = l.b - l.a;
    return l.a + v * (dot(p - l.a, v) / abs2(v));
} // 554901

```

9.2 Sweep And Polar Sort [808d8e]

```

// sort(ps.begin(), ps.end(), SweepCmp(l));
// set<P, SweepCmp> s{SweepCmp(l)};
struct SweepCmp {
    // 以 l 為掃描線，點從最左邊排到最右邊 (dir: -1 -> 1)
    P d; // 同一條掃描線上的點，依 l 的方向由起點排到終點
    SweepCmp(Line l) : d(l.b - l.a) {}
    bool operator()(P p, P q) const {
        int c = sign(cross(d, p - q));
        return c != 0 ? c > 0 : sign(dot(d, p - q)) < 0;
    }
}; // 17f9fd
struct PolarCmp {
    // 以 l.a 為中心，從 l 的方向開始轉，角度範圍 [0, 2 * PI)
    P o, d;
    int s;
    PolarCmp(Line l, int dir = 0) // dir = 0: CCW, 1: CW
        : o(l.a), d(l.b - l.a), s(dir ? -1 : 1) {}
    int half(P p) const { // 中心 -> 0, [0, PI) -> 1, [PI, 2 * PI) -> 2
        int c = sign(cross(d, p)) * s, t = sign(dot(d, p));
        return c == 0 && t == 0 ? 0 : c < 0 || (c == 0 && t < 0) ? 2 : 1;
    }
    bool operator()(P p, P q) const {
        p = p - o, q = q - o;
        if (half(p) != half(q)) return half(p) < half(q);
        int c = sign(cross(p, q)) * s;
        // 1: 同 set 視為相等, pbds_multiset >= (不管順序)
        return c > 0;
        // 2: 同點視為相等，距離近的排前面, pbds_multiset <=
        return c != 0 ? c > 0 : abs2(p) < abs2(q);
    }
}; // 1: 77c30c, 2: 73c8dc

```

9.3 Convex Hull [3413a6]

```

vector<P> convexHull(vector<P> a) {
    sort(a.begin(), a.end());
    a.resize(unique(a.begin(), a.end()) - a.begin());
    if (a.size() <= 1) return a;
    vector<P> h(a.size() * 2);
    int s = 0, t = 0;
    for (int i = 0; i < 2; i++, s = -t) {
        for (P p : a) {
            while (t >= s + 2 && sign(
                cross(h[t - 1] - h[t - 2], p - h[t - 2])) <= 0) t--;
            h[t++] = p;
        }
        reverse(a.begin(), a.end());
    }
    return {h.begin(), h.begin() + t};
}

```

9.4 Distance [039d8a]

```

double distPL(P p, Line l) {
    return abs(cross(l.a - l.b, l.a - p)) / abs(l);
} // 501648
double distancePS(P p, Line l) {
    if (sign(dot(p - l.a, l.b - l.a)) < 0) return abs(p - l.a);
    if (sign(dot(p - l.b, l.a - l.b)) < 0) return abs(p - l.b);
    return distPL(p, l);
} // ffe672
double distanceSS(Line l1, Line l2) {
    if (get<0>(segmentIntersection(l1, l2)) != 0) return 0.0;
    return min({distancePS(l1.a, l2), distancePS(
        l1.b, l2), distancePS(l2.a, l1), distancePS(l2.b, l1)});
} // fa7723

```

9.5 Point In Polygon [d1a2ce]

```

bool pointInPolygon(P a, const vector<P> &p) {
    int n = p.size(), t = 0;
    for (int i = 0; i < n; i++)
        if (btw(a, {p[i], p[(i + 1) % n]})) return true;
    for (int i = 0; i < n; i++) {
        P u = p[i], v = p[(i + 1) % n];
        if (u.x < a.x && v.x >= a.x && dir(a, {u, v}) < 0) t ^= 1;
        if (u.x >= a.x && v.x < a.x && dir(a, {u, v}) < 0) t ^= 1;
    }
    return t == 1;
}

```

9.6 Point In Convex Polygon [1e0080]

```

// 0 : strictly outside
// 1 : on boundary
// 2 : strictly inside
int pointInConvexPolygon(P a, const vector<P> &p) {
    int n = p.size();
    if (n == 0) return 0;

```

```

else if (n <= 2) return btw(a, {p[0], p.back()});
if (btw
    (a, {p[0], p[1]} || btw(a, {p[0], p[n - 1]})) return 1;
else if (dir(a, {p[0],
    p[1]} < 0 || dir(a, {p[0], p[n - 1]} < 0) return 0;
int lo = 1, hi = n - 2;
while (lo < hi) {
    int x = (lo + hi + 1) / 2;
    if (dir(a, {p[0], p[x]} < 0) lo = x;
    else hi = x - 1;
}
if (dir(a, {p[lo], p[lo + 1]} < 0) return 2;
else return btw(a, {p[lo], p[lo + 1]});
}

```

9.7 Line Intersects Polygon [254956]

```

bool lineIntersectsPolygon(Line l, const vector<P> &p) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        P a = p[i], b = p[(i + 1) % n];
        if (dir(a, l) == 0 || dir(b, l) == 0) return true;
        if ((dir(a, l) < 0) ^ (dir(b, l) < 0)) return true;
    }
    return false;
}

```

9.8 Segment Intersection [ff1cf8]

```

// 0 : not intersect
// 1 : strictly intersect
// 2 : overlap
// 3 : intersect at endpoint
tuple<int, P, P> segmentIntersection(Line l1, Line l2) {
    if (parallel(l1, l2)) {
        if (dir(l1.a, l2) != 0) return {0, {}, {}};
        else {
            vector<P> res;
            if (btw(l1.a, l2)) res.push_back(l1.a);
            if (btw(l1.b, l2)) res.push_back(l1.b);
            if (btw(l2.a, l1)) res.push_back(l2.a);
            if (btw(l2.b, l1)) res.push_back(l2.b);
            if (res.empty()) return {0, {}, {}};
            sort(res.begin(), res.end());
            res.resize(unique(res.begin(), res.end()) - res.begin());
            if (res.size() == 1) return {3, res[0], res[0]};
            else return {2, res.front(), res.back()};
        }
    }
    auto cp1 = dir(l2.b, l1), cp2 =
        dir(l2.a, l1), cp3 = dir(l1.b, l2), cp4 = dir(l1.a, l2);
    if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) ||
        (cp3 > 0 &&
            cp4 > 0) || (cp3 < 0 && cp4 < 0)) return {0, {}, {}};
    P p = lineIntersection(l1, l2);
    if (cp1 !=
        0 && cp2 != 0 && cp3 != 0 && cp4 != 0) return {1, p, p};
    else return {3, p, p};
}

```

9.9 Segment In Polygon [e28323]

```

bool segmentInPolygon(Line l, const vector<P> &p) {
    int n = p.size();
    vector<P> a{l.a, l.b};
    for (int i = 0; i < n; i++) {
        auto [t, p1,
            p2] = segmentIntersection(l, {p[i], p[(i + 1) % n]});
        if (t == 1 || t == 3) a.push_back(p1);
        else if (t == 2) a.push_back(p1), a.push_back(p2);
    }
    sort(a.begin(), a.end());
    for (int i = 0; i + 1 < a.size(); i++) {
        if (sign(abs(a[i] - a[i + 1])) == 0) continue;
        if (!pointInPolygon
            ((a[i] + a[i + 1]) / 2, p)) return false;
    }
    return true;
}

```

9.10 Half Plane Intersection [24a100]

```

// non-strict
-> >= 都改 >, assert 拿掉, 在外面特判直線(area == 0)
vector<P> hp(vector<Line> lines) {
    auto sgn = [](P p) {
        return p.y > 0 || (p.y == 0 && p.x > 0) ? 1 : -1;
    };
    sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
        auto d1 = l1.b - l1.a;
        auto d2 = l2.b - l2.a;
        if (sgn(d1) != sgn(d2)) return sgn(d1) == 1;
        return cross(d1, d2) > 0;
    });
    deque<Line> ls;
    deque<P> ps;
    for (auto l : lines) {
        if (ls.empty()) {
            ls.push_back(l);
            continue;
        }
        while (!ps.empty() &&
            dir(ps.back(), l) >= 0) ps.pop_back(), ls.pop_back();
    }
}

```

```

while (!ps.empty()
    && dir(ps[0], l) >= 0) ps.pop_front(), ls.pop_front();
if (sign(cross(
    l.b - l.a, ls.back().b - ls.back().a) == 0) { // 平行
    if (sign(dot(
        l.b - l.a, ls.back().b - ls.back().a) > 0) { // 同向
        if (dir(ls.back().a, l) >= 0) { // l 在內側
            assert(ls.size() == 1);
            ls[0] = l;
        }
        continue;
    }
    return {};
}
ps.push_back(lineIntersection(ls.back(), l));
ls.push_back(l);
}
while (!ps.empty() && dir(ps.back(), ls[0]) >= 0)
    ps.pop_back(), ls.pop_back();
if (ls.size() <= 2) return {};
ps.push_back(lineIntersection(ls[0], ls.back()));
return vector(ps.begin(), ps.end());
}

```

9.11 Min Euclidean Distance [723df0]

```

void solve(int n, vector<P> &a) { // recursive solution
    sort(a.begin(), a.end());
    vector<P> t(n);
    cout << [&](this auto &&self, int l, int r) -> ll {
        if (l == r) return 9E18;
        int p = 0, m = (l + r) / 2;
        ll mid = a[m].x;
        ll ans = min(self(l, m), self(m + 1, r));
        merge(a.begin() + l, a.begin() + m + 1,
            a.begin() + m + 1, a.begin() + r + 1,
            t.begin(),
            [](const P &a, const P &b) { return a.y < b.y; });
        copy(t.begin(), t.begin() + r - l + 1, a.begin() + l);
        for (int i = l; i <= r; i++)
            if ((mid
                - a[i].x) * (mid - a[i].x) <= ans) t[p++] = a[i];
        for (int i = 0; i < p; i++) {
            for (int j = i + 1; j < p; j++) {
                if ((t[i].y - t[j].y) * (t[i].y - t[j].y) > ans) break;
                ans = min(ans, abs2(t[i] - t[j]));
            }
        }
        return ans;
    } (0, n - 1) << "\n";
}
// K-D tree solution
struct Info {
    static constexpr int DIM = 2;
    array<ll, DIM> x, L, R;
    ll distl, distr;
    ll f(const Info &i) {
        ll ret = 0;
        if (i.
            L[0] > x[0]) ret += (i.L[0] - x[0]) * (i.L[0] - x[0]);
        if (i.
            R[0] < x[0]) ret += (x[0] - i.R[0]) * (x[0] - i.R[0]);
        if (i.
            L[1] > x[1]) ret += (i.L[1] - x[1]) * (i.L[1] - x[1]);
        if (i.
            R[1] < x[1]) ret += (x[1] - i.R[1]) * (x[1] - i.R[1]);
        return ret;
    }
};
void pull(const Info &l, const Info &r) {
    distl = f(l), distr = f(r);
};
struct KDTree { // 1-indexed
    static constexpr int DIM = Info::DIM;
    int n, rt;
    vector<Info> info;
    vector<int> l, r;
    KDTree(const vector<Info> &
        info) : n(info.size()), info(info), l(n + 1), r(n + 1) {
        rt = build(1, n);
    }
    void pull(int p) {
        info[p].L = info[p].R = info[p].x;
        info[p].pull(info[l[p]], info[r[p]]);
        for (int ch : {l[p], r[p]}) {
            if (!ch) continue;
            for (int k = 0; k < DIM; k++) {
                info[p].L[k] = min(info[p].L[k], info[ch].L[k]);
                info[p].R[k] = max(info[p].R[k], info[ch].R[k]);
            }
        }
    }
    int build(int l, int r) {
        if (r == l) return 0;
        int m = (l + r) / 2;
        array<double, DIM> av = {}, va = {};
        for (int i = l; i < r; i++)
            for (int d = 0; d < DIM; d++)
                av[d] += info[i].x[d];
        for (int d = 0; d < DIM; d++)
            av[d] /= (double)(r - l);
        for (int i = l; i < r; i++)
    }
}

```

```

    for (int d = 0; d < DIM; d++)
        va[d] +=
            (info[i].x[d] - av[d]) * (info[i].x[d] - av[d]);
    int dep = max_element(va.begin(), va.end()) - va.begin();
    nth_element
        (info.begin() + l, info.begin() + m, info.begin() + r,
        [&](const Info
            &x, const Info &y) { return x.x[dep] < y.x[dep]; });
    this->l[m] = build(l, m);
    this->r[m] = build(m + 1, r);
    pull(m); return m;
}
ll ans = 9E18;
ll dist(int a, int b) {
    return (info[a].
        x[0] - info[b].x[0]) * (info[a].x[0] - info[b].x[0]) +
        (info[a].
        x[1] - info[b].x[1]) * (info[a].x[1] - info[b].x[1]);
}
void query(int p, int x) {
    if (!p) return;
    if (p != x) ans = min(ans, dist(x, p));
    ll distl = info[x].f(info[l[p]]);
    ll distr = info[x].f(info[r[p]]);
    if (distl < ans && distr < ans) {
        if (distl < distr) {
            query(l[p], x);
            if (distr < ans) query(r[p], x);
        } else {
            query(r[p], x);
            if (distl < ans) query(l[p], x);
        }
    } else {
        if (distl < ans) query(l[p], x);
        if (distr < ans) query(r[p], x);
    }
}
}
};

```

9.12 Max Euclidean Distance [7e3b52]

```

tuple<ll, int, int> maxEuclideanDistance(vector<P> a) {
    auto get = [&](P p, Line l) -> ll
    { return abs(cross(l.a - l.b, l.a - p)); };
    ll res = 0; int n = a.size(), x, y, id = 2;
    a.push_back(a.front());
    if (n == 1) return {0, 0, 0};
    if (n == 2) return {abs2(a[0] - a[1]), 0, 1};
    for (int i = 0; i < n; i++) {
        while (get(a[id], {a[i],
            a[i + 1]}) <= get(a[(id + 1) % n], {a[i], a[i + 1]}))
            id = (id + 1) % n;
        if (res < abs2(a[i] - a[id])) {
            res = abs2(a[i] - a[id]);
            x = i, y = id;
        }
        if (res < abs2(a[i + 1] - a[id])) {
            res = abs2(a[i + 1] - a[id]);
            x = i + 1, y = id;
        }
    }
    return {res, x % n, y};
}

```

9.13 Min Circle Cover [96a4b7]

```

pair<double, P> minCircleCover(int n, vector<P> a) {
    shuffle(a.begin(), a.end(), rng);
    P c = a[0]; double r = 0;
    for (int
        i = 1; i < n; i++) if (sign(abs(c - a[i]) - r) > 0) {
        c = a[i], r = 0;
        for (int
            j = 0; j < i; j++) if (sign(abs(c - a[j]) - r) > 0) {
            c = (a[i] + a[j]) / 2.0;
            r = abs(c - a[i]);
            for (int
                k = 0; k < j; k++) if (sign(abs(c - a[k]) - r) > 0) {
                P p = (a[j] + a[i]) / 2;
                P q = (a[j] + a[k]) / 2;
                if (parallel({a[i], a[j]}, {a[j], a[k]})) continue;
                c = lineIntersection({p,
                    p + rot(a[j] - a[i])}, {q, q + rot(a[k] - a[j])});
                r = abs(c - a[i]);
            }
        }
    }
    return {r, c};
}

```

9.14 Min Rectangle Cover [3a0f29]

```

pair<double, vector<P>> minRectangleCover(vector<P> p) {
    if (p.size() <= 2) return {0, {}};
    auto get = [&](P p, Line l) -> double {
        return abs(cross(l.a - l.b, l.a - p));
    }; // line 到 p 圍成的四邊形面積
    int n = p.size(), j = 2, l = 1, r = 1;
    p.push_back(p.front());
    double ans = 8E18;
    vector<P> ps;
    for (int i = 0; i < n; i++) {

```

```

        while (get(p[j], {p[i],
            p[i + 1]}) <= get(p[(j + 1) % n], {p[i], p[i + 1]}))
            j = (j + 1) % n;
        while (dot(p[i + 1] - p[i], p[r] -
            p[i]) <= dot(p[i + 1] - p[i], p[(r + 1) % n] - p[i]))
            r = (r + 1) % n;
        if (i == 0) l = j;
        while (dot(p[i + 1] - p[i], p[l] -
            p[i]) >= dot(p[i + 1] - p[i], p[(l + 1) % n] - p[i]))
            l = (l + 1) % n;
        double area = get(p[j], {p[i], p[i + 1]});
        double w = dot(p[i] - p[i +
            1], p[l] - p[i]) + dot(p[i + 1] - p[i], p[r] - p[i]);
        area *= w / abs2(p[i + 1] - p[i]);
        if (area < ans) {
            ps.clear(), ans = area;
            Line l1(p[i], p[i + 1]);
            for (auto u : {p[r], p[j], p[l], p[i]}) {
                if (u == l1.b) {
                    ps.push_back(u);
                    l1 = {u, u + rot(l1.b - l1.a)};
                } else {
                    Line l2 = {u, u + rot(l1.b - l1.a)};
                    P res = lineIntersection(l1, l2);
                    ps.push_back(res);
                    l1 = {res, u};
                }
            }
        }
    }
    return {ans, ps};
}

```

9.15 Convex Polygon Distance [92874d]

```

// CCW needed, O(n + m)
double convexPolyDist(vector<P> a, vector<P> b) {
    int n = a.size(), m = b.size();
    int sa = 0, sb = 0;
    for (int i = 1; i < n; i++) if (a[i].y < a[sa].y) sa = i;
    for (int i = 1; i < m; i++) if (b[i].y > b[sb].y) sb = i;
    double ans = 1E18;
    for (int i = 0; i < n; i++) {
        P na = a[sa], nb = a[(sa + 1) % n];
        P ea = nb - na;
        while (sign(cross(ea, b[(sb + 1) % m] - b[sb])) > 0)
            sb = (sb + 1) % m;
        chmin(ans, distanceSS({na, nb}, {b[sb], b[(sb + 1) % m]}));
        sa = (sa + 1) % n;
    }
    return ans;
}

```

9.16 Lattice Points [fbcbcc]

```

// (interior, boundary)
pair<ll, ll> latticePoints(int n, vector<P> p, ll area2) {
    ll bnd = 0;
    p.push_back(p[0]);
    for (int i = 0; i < n; i++)
        bnd += gcd
            (abs(p[i + 1].x - p[i].x), abs(p[i + 1].y - p[i].y));
    return {(area2 - bnd) / 2 + 1, bnd};
}

```

9.17 Polygon Union Area [24443d]

```

double polygonUnion(vector<vector<P>> ps) { // CCW needed
    int n = ps.size();
    for (auto &v : ps) v.push_back(v[0]);
    double res = 0;
    auto seg = [&](P o, P a, P b) -> double {
        if (b.x - a.x == 0) return (o.y - a.y) / (b.y - a.y);
        return (o.x - a.x) / (b.x - a.x);
    };
    for (int pi = 0; pi < n; pi++) {
        for (int i = 0; i + 1 < ps[pi].size(); i++) {
            vector<pair<double, int>> e{{0, 0}, {1, 0}};
            for (int pj = 0; pj < n; pj++) {
                if (pi == pj) continue;
                for (int j = 0; j + 1 < ps[pj].size(); j++) {
                    auto c1 = cross(ps
                        [pi][i + 1] - ps[pi][i], ps[pj][j] - ps[pi][i]);
                    auto c2 = cross(ps[pi][
                        i + 1] - ps[pi][i], ps[pj][j + 1] - ps[pi][i]);
                    if (c1 == 0 && c2 == 0) {
                        if (dot(ps[pi][i + 1] - ps[pi][i], ps[pj]
                            ][j + 1] - ps[pj][j]) > 0 && (pi - pj) > 0) {
                            e.emplace_back(seg
                                (ps[pj][j], ps[pi][i], ps[pi][i + 1]), 1);
                            e.emplace_back(seg(ps
                                [pj][j + 1], ps[pi][i], ps[pi][i + 1]), -1);
                        }
                    } else {
                        auto s1 = cross(ps[pj]
                            ][j + 1] - ps[pj][j], ps[pi][i] - ps[pj][j]);
                        auto s2 = cross(ps[pj][j
                            ] + 1] - ps[pj][j], ps[pi][i + 1] - ps[pj][j]);
                        if (c1 >= 0
                            && c2 < 0) e.emplace_back(s1 / (s1 - s2), 1);
                        else if (c1 < 0 &&
                            c2 >= 0) e.emplace_back(s1 / (s1 - s2), -1);

```

```

    }
}
sort(e.begin(), e.end());
double pre = clamp(e[0].first, 0.0, 1.0), sum = 0;
int cov = e[0].second;
for (int j = 1; j < e.size(); j++) {
    double now = clamp(e[j].first, 0.0, 1.0);
    if (!cov) sum += now - pre;
    cov += e[j].second;
    pre = now;
}
res += cross(ps[pi][i], ps[pi][i + 1]) * sum;
}
return res / 2;
}

```

10 Polynomial

10.1 FFT [569b66]

```

const double PI = acos(-1.0);
using cd = complex<double>;
void fft(vector<cd> &a, bool inv = false) {
    static vector<cd> w(0, 1);
    int n = a.size();
    for (int k = w.size(); k < n; k *= 2) {
        w.resize(2 * k);
        for (int j = 0; j < k; j++) w[k + j] = polar(1.0, PI * j / k);
    }
    if (!inv) {
        for (int k = n / 2; k >= 1; k /= 2)
            for (int i = 0; i < n; i += 2 * k)
                for (int j = 0; j < k; j++) {
                    cd u = a[i + j], v = a[i + j + k];
                    a[i + j] = u + v;
                    a[i + j + k] = (u - v) * w[k + j];
                }
        return;
    }
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                cd u = a[i + j], v = a[i + j + k] * w[k + j];
                a[i + j] = u + v;
                a[i + j + k] = u - v;
            }
    reverse(a.begin() + 1, a.end());
    for (auto &x : a) x /= n;
}
// i + j = k, res[1] = a[1] * b[0] + a[0] * b[1]
// reverse b for i - j = k, res[k + b.size() - 1]
template
<class T> vector<double> conv(vector<T> a, vector<T> b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1, tot = a.size() + b.size() - 1;
    while (n < tot) n <= 1;
    fa.resize(n), fb.resize(n);
    fft(fa), fft(fb);
    for (int i = 0; i < n; i++) fa[i] = fa[i] * fb[i];
    fft(fa, true);
    vector<double> res(tot);
    for (int i = 0; i < tot; i++) res[i] = fa[i].real();
    return res; // use lround to int
}

```

10.2 NTT [0eaf84]

```

template<class Z>
void ntt(vector<Z> &a, bool inv = false, const int G = 3) {
    static vector<Z> w(0, 1);
    int n = a.size();
    if (w.size() < n) {
        int k = __builtin_ctz(w.size());
        w.resize(n);
        while ((1 << k) < n) {
            Z u = power(Z(G), (Z::p() - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < 1 << k; i++) {
                w[i * 2] = w[i];
                w[i * 2 + 1] = w[i] * u;
            }
            k++;
        }
    }
    if (inv) {
        for (int k = 1; k < n; k *= 2) {
            for (int i = 0; i < n; i += 2 * k) {
                for (int j = 0; j < k; j++) {
                    Z u = a[i + j];
                    Z v = a[i + j + k] * w[k + j];
                    a[i + j] = u + v;
                    a[i + j + k] = u - v;
                }
            }
        }
        reverse(a.begin() + 1, a.end());
        Z inv = Z(n).inv();
        for (auto &x : a) x *= inv;
    } else {
        for (int k = n / 2; k >= 1; k /= 2) {

```

```

            for (int i = 0; i < n; i += 2 * k) {
                for (int j = 0; j < k; j++) {
                    Z u = a[i + j];
                    Z v = a[i + j + k];
                    a[i + j] = u + v;
                    a[i + j + k] = (u - v) * w[k + j];
                }
            }
        }
    } // d2e699
template<class Z>
vector<Z> conv(vector<Z> a, vector<Z> b, const int G = 3) {
    if (a.empty() || b.empty()) return {};
    int n = 1, tot = a.size() + b.size() - 1;
    while (n < tot) n <= 1;
    a.resize(n), b.resize(n);
    ntt(a, false, G), ntt(b, false, G);
    for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
    ntt(a, true, G);
    a.resize(tot);
    return a;
} // 7866d6
const int G = 3;
const int P1 = 167772161, P2 = 104857601, P3 = 998244353;
using Z1 = Mint<P1>;
using Z2 = Mint<P2>;
using Z3 = Mint<P3>;
const int I12 = Z2(P1).inv().x;
const int I123 = Z3(1LL * P1 * P2).inv().x;
template<class T> T garner(int r1, int r2, int r3) {
    Z2 t1 = Z2(r2 - r1) * I12;
    Z2 t2 = Z3(r3 - r1 - 1LL * P1 * t1.x) * I123;
    return T(r1) + T(P1) * t1.x + T(P1) * P2 * t2.x;
} // 6def25
template<class Z>
vector<Z> convArb(vector<Z> a, vector<Z> b) {
    vector<Z1> a1(a.size()), b1(b.size());
    vector<Z2> a2(a.size()), b2(b.size());
    vector<Z3> a3(a.size()), b3(b.size());
    for (int i = 0; i < a.size(); i++) {
        a1[i] = a[i].x;
        a2[i] = a[i].x;
        a3[i] = a[i].x;
    }
    for (int i = 0; i < b.size(); i++) {
        b1[i] = b[i].x;
        b2[i] = b[i].x;
        b3[i] = b[i].x;
    }
    auto c1 = conv(a1, b1);
    auto c2 = conv(a2, b2);
    auto c3 = conv(a3, b3);
    int tot = a.size() + b.size() - 1;
    vector<Z> res(tot);
    for (int i = 0; i < tot; i++)
        res[i] = garner<Z>(c1[i].x, c2[i].x, c3[i].x);
    return res;
} // 7e3465

```

10.3 FWT [c53534]

```

template<class T> void fwt(vector<T> &a, bool inv = false) {
    // Or : [[1, 0], [1, 1]] <-> [[1, 0], [-1, 1]]
    // And: [[1, 1], [0, 1]] <-> [[1, -1], [0, 1]]
    // Xor: [[1, 1], [1, -1]] <-> [[0.5, 0.5], [0.5, -0.5]]
    // *inv_2 instead of /2 for Z
    int n = __lg(int(a.size()));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 1 << n; j++) {
            if (j >> i & 1) { // And
                T x = a[j ^ (1 << i)];
                T y = a[j];
                if (!inv) {
                    a[j ^ (1 << i)] = x + y;
                    a[j] = y;
                } else {
                    a[j ^ (1 << i)] = x - y;
                    a[j] = y;
                }
            }
        }
    }
}

```

10.4 Polynomial [cf9756]

```

struct Poly : public vector<Mint<P>> {
    using Z = Mint<P>;
    explicit Poly(int n = 0) : vector<Z>(n) {}
    Poly(const vector<Z> &a) : vector<Z>(a) {}
    Poly(const initializer_list<Z> &a) : vector<Z>(a) {}
    template<class InputIt, class = _RequireInputIter<InputIt>>
    Poly(InputIt first, InputIt last) : vector<Z>(first, last) {}
    Poly operator-() const {
        vector<Z> res(this->size());
        for (int i = 0; i < int(res.size()); i++) res[i] = -(*this)[i];
        return Poly(res);
    }
    friend Poly operator+(Poly a, Poly b) {
        a.resize(max(a.size(), b.size()));

```

```

    for (int i = 0; i < b.size(); i++) a[i] += b[i];
    return a;
}
friend Poly operator-(Poly a, Poly b) {
    a.resize(max(a.size(), b.size()));
    for (int i = 0; i < b.size(); i++) a[i] -= b[i];
    return a;
}
friend Poly operator*(Poly a, Poly b) {
    return conv(a, b, G);
}
friend Poly operator*(Poly a, Z x) {
    for (int i = 0; i < a.size(); i++) a[i] *= x;
    return a;
}
friend Poly operator/(Poly a, Z x) {
    for (int i = 0; i < a.size(); i++) a[i] /= x;
    return a;
}
Poly &operator+=(Poly a) { return *this = *this + a; }
Poly &operator-=(Poly a) { return *this = *this - a; }
Poly &operator*=(Poly a) { return *this = *this * a; }
Poly &operator*=(Z a) { return *this = *this * a; }
Poly &operator/=(Z a) { return *this = *this / a; }
// 7a07f6
Poly shift(int k) const {
    if (k >= 0) {
        auto b = *this;
        b.insert(b.begin(), k, 0);
        return b;
    } else if (this->size() <= -k) {
        return Poly();
    } else {
        return Poly(this->begin() + (-k), this->end());
    }
}
// e6ab95
Poly trunc(int k) const {
    Poly f = *this; f.resize(k); return f;
}
// 217dbc
Poly deriv() const {
    if (this->empty()) return Poly();
    Poly res(this->size() - 1);
    for (int i = 0; i < this->size() - 1; i++)
        res[i] = (*this)[i + 1] * (i + 1);
    return res;
}
// ccea0
Poly integr() const {
    Poly res(this->size() + 1);
    for (int i = 0; i < this->size(); i++)
        res[i + 1] = (*this)[i] / (i + 1);
    return res;
}
// f86ffe
Poly inv(int m) const {
    Poly x((*this)[0].inv());
    int k = 1;
    while (k < m) {
        k *= 2;
        x = (x * (Poly{2} - trunc(k) * x)).trunc(k);
    }
    return x.trunc(m);
}
// 8287e6
Poly log(int m) const {
    return (deriv() * inv(m)).integr().trunc(m);
}
// bfd979
Poly pow(ll k, int m) const {
    if (k == 0) { Poly res(m); res[0] = 1; return res; }
    int i = 0;
    while (i < this->size() && (*this)[i].x == 0) i++;
    if (i == this->size() || i > 0 && k > (m - 1) / i) return Poly(m);
    Z v = (*this)[i];
    auto f = shift(-i) * v.inv();
    return (f.log(m - i * k) * Z(k).exp(m - i * k).shift(i * k) * power(v, k));
}
// 23cb76
Poly sqrt(int m) const { // need quadraticResidue
    int k = 0;
    while (k
        < this->size() && (*this)[k].x == 0) k++; // 找前導零
    if (k == this->size()) return Poly(m); // 全零多項式
    if (k % 2 != 0) return Poly(); // 無解: 最低次項為奇數
    int s = quadraticResidue((*this)[k]);
    if (s == -1) return Poly(); // 無解: 係數無平方根
    int ofst = k / 2, r = m - ofst;
    if (r <= 0) return Poly(m);
    Poly h = this->shift(-k) * (*this)[k].inv(), g[1];
    for (int i = 1; i < r; i <= 1) {
        int len = i < 1;
        g = (g + h.trunc(len) * g.inv(len)).trunc(len) / 2;
    }
    g.resize(r);
    g = (g * Z(s)).shift(ofst).trunc(m);
    return g;
}
// 1a3aea
Poly exp(int m) const {
    Poly x[1];
    int k = 1;
    while (k < m) {
        k *= 2;
        x = (x * (Poly{1} - x.log(k) + trunc(k))).trunc(k);
    }
}

```

```

    return x.trunc(m);
}
// 78baba
Poly mult(Poly b) const {
    if (b.empty()) return Poly();
    int n = b.size();
    reverse(b.begin(), b.end());
    return ((*this) * b).shift(-(n - 1));
}
// f7c257
vector<Z> eval(vector<Z> x) const {
    if (this->size() == 0) return vector<Z>(x.size(), 0);
    const int n = max(x.size(), this->size());
    vector<Poly> q(4 * n);
    vector<Z> ans(x.size());
    x.resize(n);
    function<void>
        (int, int, int) build = [&](int p, int l, int r) {
        if (r - l == 1) {
            q[p] = Poly{1, -x[l]};
        } else {
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            q[p] = q[2 * p] * q[2 * p + 1];
        }
    };
    build(1, 0, n);
    function<void>(int, int, int, const Poly
        &>) work = [&](int p, int l, int r, const Poly &num) {
        if (r - l == 1) {
            if (l < int(ans.size())) ans[l] = num[0];
        } else {
            int m = (l + r) / 2;
            work(2 * p, l, m, num.mult(q[2 * p + 1]).trunc(m - l));
            work(2 * p + 1, m, r, num.mult(q[2 * p]).trunc(r - m));
        }
    };
    work(1, 0, n, mult(q[1].inv(n)));
    return ans;
}
// 9698d5
}
}

```

10.5 Barlekamp Massey [d6813f]

```

// find s[i] = c[0]
// * s[i - 1] + c[1] * s[i - 2] + ... + c[k - 1] * s[i - k]
// return : c(x) = 1 - c_0x - c_1x^2 - ... - c_{k-1}x^k
Poly berlekampMassey(const Poly &s) { // 0(n^2)
    Poly c, oldC;
    int f = -1;
    for (int i = 0; i < s.size(); i++) {
        auto delta = s[i];
        for (int j = 1; j <= c.size(); j++)
            delta -= c[j - 1] * s[i - j];
        if (delta.x == 0) continue;
        if (f == -1) {
            c.resize(i + 1);
            f = i;
        } else {
            auto d = oldC;
            d *= -1;
            d.insert(d.begin(), 1);
            Z df1 = 0;
            for (int j = 1; j <= d.size(); j++)
                df1 += d[j - 1] * s[f + 1 - j];
            assert(df1.x != 0);
            auto coef = delta / df1;
            d *= coef;
            Poly zeros(i - f - 1);
            zeros.insert(zeros.end(), d.begin(), d.end());
            d = zeros;
            auto temp = c;
            c += d;
            if (i - temp.size() > f - oldC.size()) {
                oldC = temp;
                f = i;
            }
        }
    }
    c *= -1;
    c.insert(c.begin(), 1);
    return c;
}

```

10.6 Linear Recurrence [81be8a]

```

// a_i = c[0] *
// a_{i - 1} + c[1] * a_{i - 2} + ... + c[k - 1] * a_{i - k}
// q(x) = 1 - c[0] * x - c[1] * x^2 - ... - c[k - 1] * x^k
// p(x) = (a * q).trunc(c.size())
// return : a_n
Z linearRecurrence(Poly p, Poly q, ll n) { // 0(m log m log n)
    int m = q.size() - 1;
    while (n > 0) {
        auto nq = q;
        for (int i = 1; i <= m; i += 2) nq[i] *= -1;
        auto np = p * nq;
        nq = q * nq;
        for (int i = 0; i < m; i++) p[i] = np[i * 2 + n % 2];
        for (int i = 0; i <= m; i++) q[i] = nq[i * 2];
        n /= 2;
    }
    return p[0] / q[0];
}

```

Prime	Root	Prime	Root
7681	17	167772161	3
12289	11	104857601	3
40961	3	985661441	3
65537	3	998244353	3
786433	10	1107296257	10
5767169	3	2013265921	31
7340033	3	2810183681	11
23068673	3	2885681153	3
469762049	3	605028353	3
2061584302081	7	194555039024054273	5
2748779069441	3	9223372036737335297	3

11 Else

11.1 Segment Manipulation [30596f]

```

auto era = [&](set<pair<int, int>> &st, int L, int R) {
    auto it = st.lower_bound({L, -1});
    if (it != st.begin() && prev(it)->second >= L) --it;
    while (it != st.end() && it->first <= R) {
        auto [l, r] = *it;
        it = st.erase(it);
        if (l < L) st.emplace(l, L - 1);
        if (R < r) st.emplace(R + 1, r);
    }
};
// remove + 1 for strictly overlap
auto ins = [&](set<pair<int, int>> &st, int L, int R) {
    auto it = st.lower_bound({L, -1});
    if (it != st.begin() && prev(it)->second + 1 >= L) --it;
    while (it != st.end() && it->first <= R + 1) {
        L = min(L, it->first);
        R = max(R, it->second);
        it = st.erase(it);
    }
    st.emplace(L, R);
};
auto find = [&](const set<pair<int, int>> &st, int i) {
    auto it = st.lower_bound({i + 1, -1});
    if (it != st.begin()) {
        --it;
        if (it->second >= i) return it;
    }
    return st.end();
};

```

11.2 Python [7c66a4]

```

from decimal import * # 高精度浮點數
from fractions import * # 分數
from random import *
from math import *
# set decimal prec bigger if it could overflow in precision
setcontext(
    Context(prec=10, Emax=MAX_EMAX, rounding=ROUND_FLOOR))
# read and print
x = int(input())
a, b, c = list(map(Fraction, input().split()))
arr = list(map(Decimal, input().split()))
print(*arr)
# set
st = set(); st.add((a, b)); st.remove((a, b))
if not (a, b) in st:
    # dict
    d = dict(); d[(a, b)] = 1; del d[(a, b)]
    for (a, b) in d.items():
        # random
        arr = [randint(l, r) for i in range(size)]
        choice([8, 6, 4, 1]) # random pick one
        shuffle(arr)

```

11.3 Bigint [024a84]

```

struct Bigint { // not support hex division
private:
    static const int digit = 9; // hex: 7
    static const int base = 10; // hex: 16
    static const int B = power(ll(base), digit);
    Bigint(vector<int> x, int sgn) : x(x), sgn(sgn) {}
    template<class U> vector<int> norm(vector<U> a) {
        if (a.empty()) return {0};
        for (int i = 0; i < a.size(); i++) {
            U c = a[i];
            a[i] %= B;
            if (c / B)
                if (i == a.size() - 1) a.push_back(c / B);
                else a[i + 1] += c / B;
        }
        while (a.size() > 1 && a.back() == 0) a.pop_back();
        return {a.begin(), a.end()};
    }
    void resign() { sgn = x.back() == 0 ? 1 : sgn; }
    int toInt(char c) const {
        if (isdigit(c)) return c - '0';
        else return c - 'A' + 10;
    }
    char toChar(int c) const {
        if (c < 10) return c + '0';
        else return c - 10 + 'A';
    }
}

```

```

public:
    int sgn = 1;
    vector<int> x; // 反著存
    Bigint() : x({0}), sgn(1) {}
    Bigint(ll a) { *this = Bigint(std::to_string(a)); }
    Bigint(string s) {
        if (s.empty()) { *this = Bigint(); return; }
        if (s[0] == '-') s.erase(s.begin()), sgn = -1;
        int add = 0, cnt = 0, b = 1;
        while (s.size()) {
            if (cnt == digit) x.push_back(add), add = cnt = 0, b = 1;
            add += toInt(s.back()) * b;
            cnt++, b *= base;
            s.pop_back();
        }
        if (add) x.push_back(add);
        x = norm(x);
    }
    int size() const { return x.size(); }
    Bigint abs() const { return Bigint(x, 1); }
    string to_string() const {
        string res;
        for (int i = 0; i < x.size(); i++) {
            string add; int v = x[i];
            for (int j = 0; j < digit; j++)
                add += toChar(v % base), v /= base;
            res += add;
        }
        while (res.size() > 1 && res.back() == '0') res.pop_back();
        if (sgn == -1) res += '-';
        reverse(res.begin(), res.end());
        return res;
    }
    Bigint operator-() const { return Bigint(x, -sgn); }
    friend Bigint operator+(Bigint a, Bigint b) {
        if (a.sgn != b.sgn) return a - (-b);
        int n = max(a.size(), b.size());
        a.x.resize(n), b.x.resize(n);
        for (int i = 0; i < n; i++) a.x[i] += b.x[i];
        a.x = a.norm(a.x), a.resign();
        return a;
    }
    friend Bigint operator-(Bigint a, Bigint b) {
        if (a.sgn != b.sgn) return a + (-b);
        if (a.abs() < b.abs()) return -(b - a);
        int n = max(a.size(), b.size());
        a.x.resize(n), b.x.resize(n);
        for (int i = 0; i < n; i++) {
            a.x[i] -= b.x[i];
            if (a.x[i] < 0) a.x[i] += B, a.x[i + 1]--;
        }
        a.x = a.norm(a.x), a.resign();
        return a;
    }
    friend istream &operator>>(istream &is, Bigint &a) {
        string v; is >> v; a = Bigint(v); return is; }
    friend ostream &operator<<(ostream &os, Bigint &a) {
        os << a.to_string(); return os; }
    friend bool operator<(Bigint a, Bigint b) {
        if (a.sgn != b.sgn) return a.sgn < b.sgn;
        if (a.x.size() != b.x.size())
            return a.x.size() < b.x.size();
        else for (int i = a.x.size() - 1; i >= 0; i--)
            if (a.x[i] != b.x[i]) return a.x[i] < b.x[i];
        return 0;
    }
    friend bool operator>(Bigint a, Bigint b) {
        if (a.sgn != b.sgn) return a.sgn > b.sgn;
        if (a.x.size() != b.x.size())
            return a.x.size() > b.x.size();
        else for (int i = a.x.size() - 1; i >= 0; i--)
            if (a.x[i] != b.x[i]) return a.x[i] > b.x[i];
        return 0;
    }
    friend bool operator==(const Bigint &a, const Bigint &b) {
        return a.sgn == b.sgn && a.x == b.x; }
    friend bool operator!=(const Bigint &a, const Bigint &b) {
        return a.sgn != b.sgn || a.x != b.x; }
    friend bool operator>=(const Bigint &a, const Bigint &b) {
        return a == b || a > b; }
    friend bool operator<=(const Bigint &a, const Bigint &b) {
        return a == b || a < b; }
};
Bigint abs(const Bigint &a) { return a.abs(); }
Bigint stoiBigint(const string &s) { return Bigint(s); }

```

11.4 Multiple [7600ae]

```

// Require: Mint, NTT
using i128 = __int128_t;
vector<i128> arbitraryMult(
    const vector<int> &a, const vector<int> &b) {
    auto x = conv(vector<Z1>(a.begin(), a.end()), vector<Z1>(b.begin(), b.end()));
    auto y = conv(vector<Z2>(a.begin(), a.end()), vector<Z2>(b.begin(), b.end()));
    auto z = conv(vector<Z3>(a.begin(), a.end()), vector<Z3>(b.begin(), b.end()));
    vector<i128> res(x.size());
    for (int i = 0; i < x.size(); i++)
        res[i] = garner<i128>(x[i].x, y[i].x, z[i].x);
    return res;
}

```

```

}
public:
    friend Bigint operator*(Bigint a, const Bigint &b) {
        a.x = a.norm(arbitraryMult(a.x, b.x));
        a.sgn *= b.sgn, a.resign();
        return a;
    }
}

```

11.5 Division [Sbe5f1]

```

private:
    pair<vector<int>, int> smallDivMod(vector<int> a, int v) {
        ll add = 0;
        for (int i = a.size() - 1; i >= 0; i--) {
            add = add * B + a[i];
            int q = add / v;
            a[i] = q, add %= v;
        }
        return {norm(a), add};
    }
    friend Bigint operator<<(Bigint a, int k) {
        if (!a.x.empty()) {
            vector<int> add(k, 0);
            a.x.insert(a.x.begin(), add.begin(), add.end());
        }
        return a;
    }
    friend Bigint operator>>(Bigint a, int k) {
        a.x = vector<int>
            >(a.x.begin() + min(k, int(a.x.size())), a.x.end());
        a.x = a.norm(a.x);
        return a;
    }
public:
    friend pair<Bigint
        , Bigint> divmod(const Bigint &a, const Bigint &b) {
        int qsgn = a.sgn * b.sgn, rsgn = a.sgn;
        Bigint av = a.abs(), bv = b.abs();
        Bigint quot, rem;
        if (av < bv) {
            rem = av;
        } else if (bv.size() == 1) {
            auto [qd, r] = quot.smallDivMod(av.x, bv.x[0]);
            quot.x = qd;
            rem = Bigint((ll)r);
        } else {
            Bigint inv = 1LL * B * B / bv.x.back();
            Bigint pre = 0, res = 0;
            int d = av.size() + 1 - bv.size();
            int cur = 2, bcur = 1;
            while (inv != pre || bcur < bv.size()) {
                bcur = min(bcur << 1, bv.size());
                res.x = {bv.x.end() - bcur, bv.x.end()};
                pre = inv;
                inv = inv
                    * ((Bigint(2) << (cur + bcur - 1)) - inv * res);
                cur = min(cur << 1, d);
                inv.x = {inv.x.end() - cur, inv.x.end()};
            }
            inv.x = {inv.x.end() - d, inv.x.end()};
            res = (av * inv) >> av.size();
            Bigint mul = res * bv;
            while (mul + bv <= av) res = res + 1, mul = mul + bv;
            quot.x = quot.norm(res.x);
            rem = av - mul; // 重用 mul, 不再重算一次 quot * b
        }
        quot.sgn = qsgn, quot.resign();
        rem.sgn = rsgn, rem.resign();
        return {quot, rem};
    }
    friend Bigint operator/(const Bigint
        &a, const Bigint &b) { return divmod(a, b).first; }
    friend Bigint operator%(const Bigint
        &a, const Bigint &b) { return divmod(a, b).second; }
    Bigint gcd(Bigint a, Bigint b) {
        while (b != 0) {
            Bigint r = a % b;
            a = b, b = r;
        }
        return a;
    }
}

```

11.6 Division-Python [110bd8]

```

from decimal import * # 無誤差浮點數
setcontext(
    Context(prec=4000000, Emax=4000000, rounding=ROUND_FLOOR))
t = int(input())
for i in range(t):
    a, b = map(Decimal, input().split())
    d, m = divmod(a, b)
    print(d, m)

```